



INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA
DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

LICENCIATURA EM ENGENHARIA INFORMÁTICA E MULTIMÉDIA
UNIDADE CURRICULAR DE PROJETO

Sistema de gestão central para smart tool cabinets - Mobile Interface



Gonçalo Serrano Charneca (n.º 50727)

Maria Luísa Geraldês Barba de Melo e Sampaio (n.º 50799)

Orientadores

Professor Tiago Miguel Braga da Silva Dias

Professor Carlos Jorge de Sousa Gonçalves

Julho, 2026

Resumo

Em oficinas de manutenção aeronáutica, cada técnico utiliza diariamente um vasto conjunto de ferramentas, com diferentes funções, níveis de criticidade e requisitos de controlo, por isso é crítico saber onde se encontra cada ferramenta, quem a requisitou e em que intervenção foi utilizada, pois uma única ferramenta esquecida no interior de uma aeronave pode originar falhas mecânicas graves e comprometer a segurança do voo. Como este controlo é tradicionalmente feito de forma manual, torna-se demorado e vulnerável a erros.

Face a este problema, este relatório apresenta uma aplicação multiplataforma desenvolvida para apoiar a gestão centralizada de armários inteligentes de ferramentas. A aplicação disponibiliza uma interface gráfica adaptada aos diferentes perfis de utilizador — Gestor, Técnico e *Back-Office* — e comunica com uma API REST responsável por centralizar os dados e as regras de negócio do sistema. A requisição e a devolução de ferramentas ficam associadas às tarefas atribuídas a cada técnico, permitindo saber, em qualquer momento, quem tem cada ferramenta, qual o seu estado e, em que contexto foi utilizada.

Desenvolvida em *Kotlin Multiplatform* para Android e iOS, a solução integra autenticação por *email*/palavra-passe, assim como autenticação federada por *single sign-on*, e comunica com uma API REST que centraliza os dados e as regras de negócio do sistema. Com esta abordagem, a aplicação contribui para melhorar a rastreabilidade das ferramentas, reduzir a dependência de processos manuais e apoiar uma gestão mais segura e eficiente das operações de manutenção.

Palavras-chave: manutenção aeronáutica, rastreabilidade de ferramentas, smart tool cabinets, aplicações móveis, *Kotlin Multiplatform*.

Abstract

In aircraft maintenance workshops, each technician uses a wide range of tools on a daily basis, with different functions, criticality levels, and control requirements, making it critical to know where each tool is located, who requisitioned it, and in which intervention it was used, since a single tool left inside an aircraft can lead to serious mechanical failures and compromise flight safety. As this control is traditionally carried out manually, it becomes time-consuming and prone to errors.

Given this problem, this report presents a multiplatform application developed to support the centralized management of smart tool cabinets. The application provides a graphical interface adapted to different user profiles — Manager, Technician, and *Back-Office* — and communicates with a **REST API** responsible for centralizing the system’s data and business rules. Tool requisition and return are linked to the tasks assigned to each technician, making it possible to know, at any time, who has each tool, its status, and the context in which it was used.

Developed in *Kotlin Multiplatform* for Android and iOS, the solution integrates email/password authentication as well as federated single sign-on authentication, and communicates with a **REST API** that centralizes the system’s data and business rules. With this approach, the application helps improve tool traceability, reduce reliance on manual processes, and support safer and more efficient management of maintenance operations.

Keywords: aircraft maintenance, tool traceability, smart tool cabinets, mobile applications, Kotlin Multiplatform.

Agradecimentos

Gostaríamos de expressar o nosso sincero agradecimento aos coordenadores de projeto, o Professor Carlos Gonçalves e o Professor Tiago Dias, por toda a orientação, partilha de conhecimento e disponibilidade demonstrada ao longo do desenvolvimento deste projeto.

Agradecemos também a todos os docentes do ISEL que ao longo do nosso percurso académico, contribuíram para a nossa formação.

Por fim, deixamos um agradecimento especial às nossas famílias e amigos, pelo apoio incondicional, paciência e incentivo constante que nos deram durante toda esta etapa académica.

Eu, Maria Luísa Melo e Sampaio dedico este projeto à minha família e amigos, que sempre me apoiaram e acreditaram em mim ao longo destes últimos anos, mesmo quando eu não acreditava.

Eu, Gonçalo Charneca, dedico este projeto a toda a minha família, pelo apoio constante em todos os momentos e por me ter proporcionado a oportunidade de estudar e de ingressar no ensino superior, no curso que sempre quis.

Por fim, agradecemos a todos os nossos colegas, amigos e docentes do ISEL que fizeram parte desta jornada.

Índice

Resumo	i
Abstract	iii
Agradecimentos	v
Lista de Figuras	xi
Lista de Tabelas	xiii
Lista de Acrónimos	xv
1 Introdução	1
1.1 Objetivos e contributos	1
1.2 Trabalho Relacionado	2
1.3 Utilização de IA no contexto deste trabalho	3
1.4 Organização do Documento	4
2 Requisitos e modelo funcional	5
2.1 Perfis de Utilizador	5
2.2 Requisitos	6
2.2.1 Requisitos Funcionais	6
2.2.2 Requisitos Não Funcionais	9
2.2.3 Casos de Utilização	11
3 Arquitetura do sistema informático	15
3.1 Persistência e rastreabilidade de dados	15
3.2 Comunicação entre aplicação móvel e componente de servidor	16
3.3 Desenvolvimento multiplataforma da aplicação móvel	16
3.4 Organização interna da aplicação móvel	17
3.4.1 Autenticação e controlo de acesso	19
3.5 Abordagem de desenvolvimento	20
4 Implementação	21
4.1 Modelo de Dados	21
4.2 API REST	23
4.3 Aplicação móvel multiplataforma	24
4.3.1 Estrutura do Projeto	25
4.3.2 Aplicação do padrão MVVM	26

4.3.3	Construção de um Ecrã Compose	27
4.3.4	Navegação	28
4.3.5	Arranque e Gestão de Sessão	28
4.3.6	Autenticação	28
4.3.7	Injeção de Dependências e Acesso à API	29
5	Validação e Testes	31
5.1	Metodologia de validação	31
5.2	Autenticação e Controlo de acessos	31
5.3	Atribuição de tarefas e requisição de ferramentas	32
5.4	Devolução de ferramentas e Reporte de anomalias	34
6	Conclusões e Trabalho Futuro	37
	Referências bibliográficas	39
A	Gestão de Versões e Repositório	41
A.1	Ferramentas e Repositório	41
A.2	Fluxo de Trabalho	41
A.3	Convenção de Nomes dos Ramos	41
B	Protótipo da Interface (Mockups)	43
B.1	Perfil de Gestor	43
B.2	Perfil de Técnico	44
B.3	Perfil de Back-Office	44
C	Tabela dos Pontos de Acesso	45
D	Modelo Entidade Associação	47

Lista de Figuras

2.1	Diagrama de casos de utilização do <i>Back Office</i>	11
2.2	Diagrama Caso de Utilização Gestor	12
2.3	Diagrama Caso de Utilização Técnico	13
3.1	Estrutura do padrão MVVM	18
4.1	Modelo Entidade-Associação	21
4.2	Diagrama de <i>Source Sets</i> utilizados	26
5.1	Fluxo de autenticação da aplicação	32
5.2	Atribuição de tarefa e ferramentas pelo Gestor	33
5.3	Processo de requisição de ferramentas por um Técnico	34
5.4	Processo de devolução e reporte de anomalia	35
B.1	<i>Mockups</i> do perfil de Gestor.	43
B.2	<i>Mockups</i> do perfil de Técnico.	44
B.3	<i>Mockups</i> do perfil de <i>Back-Office</i>	44
D.1	Modelo Entidade-Associação	48

Lista de Tabelas

2.1	Requisitos funcionais do sistema	6
2.2	Requisitos de gestão de funcionários	7
2.3	Requisitos funcionais da interface gráfica	7
2.4	Requisitos funcionais de tarefas e ferramentas	8
2.5	Requisitos funcionais de requisição e devolução	8
2.6	Requisitos funcionais de notificação	8
2.7	Requisitos funcionais de autenticação	9
2.8	Requisitos não funcionais estéticos	9
2.9	Requisitos de facilidade de utilização	9
2.10	Requisitos não funcionais de plataformas	10
2.11	Requisitos de interação pessoa-máquina	10
2.12	Requisitos não funcionais de acessibilidade	10
4.1	Principais pontos de acesso (<i>endpoints</i>) da <i>API</i>	24
C.1	Pontos de acesso (<i>endpoints</i>) da <i>API</i>	45

Lista de Acrónimos

API *Application Programming Interface.*

CTT *Correios de Portugal.*

FOD *Foreign Object Debris.*

HTTP *Hypertext Transfer Protocol.*

IA *Inteligência Artificial.*

iOS *iPhone Operating System.*

JDBC *Java Database Connectivity.*

JSON *JavaScript Object Notation.*

KMP *Kotlin Multiplatform.*

MVC *Model-View-Controller.*

OAuth *Open Authorization.*

PIN *Personal Identification Number.*

QR *Quick Response.*

RFID *Radio-Frequency Identification.*

SSO *Single Sign-On.*

UI *User Interface.*

UX *User Experience.*

XML *Extensible Markup Language.*

Capítulo 1

Introdução

A manutenção de aeronaves é uma atividade altamente regulada, sujeita a normas rigorosas de segurança e a auditorias frequentes. Cada intervenção pode exigir a utilização de um conjunto alargado de ferramentas, com diferentes funções, níveis de criticidade e requisitos de controlo, sendo essencial garantir que, no final de cada intervenção, todas as ferramentas são contabilizadas, localizadas e devolvidas ao armário.

Uma ferramenta esquecida no interior de uma aeronave, situação conhecida na indústria aeronáutica como *Foreign Object Debris* (FOD), pode originar falhas mecânicas graves e comprometer a segurança do voo, com consequências potencialmente catastróficas. Este risco faz do controlo rigoroso das ferramentas uma prioridade nas operações de manutenção.

A este risco junta-se a necessidade de assegurar a rastreabilidade das ferramentas ao longo do tempo. Por exemplo, se for detetado posteriormente que uma ferramenta se encontrava mal calibrada, é necessário saber em que aeronaves foi usada, de forma a avaliar se essas aeronaves devem ser sujeitas a uma nova avaliação. Sem um registo fiável do histórico de utilização de cada ferramenta, esta análise torna-se difícil, lenta e propensa a erros.

Atualmente, grande parte deste controlo assenta em processos manuais, que não asseguram, de forma sistemática, a contabilização das ferramentas em tempo real, nem a reconstituição rigorosa do seu histórico de utilização. Neste contexto, torna-se relevante desenvolver uma solução digital que apoie a gestão centralizada de ferramentas, permita associar a sua utilização a técnicos e tarefas, e contribua para reduzir erros nos processos de requisição e devolução.

1.1 Objetivos e contributos

O objetivo principal deste trabalho foi o desenvolvimento de uma solução multiplataforma que apoiasse a gestão centralizada de armários inteligentes de ferramentas em contexto de manutenção aeronáutica. Pretendia-se que esta solução permitisse diferentes perfis de utilizador – Técnico, Gestor e *Back-Office* –, consultar informação operacional, apoiar os processos de requisição e devolução de ferramentas, associar ferramentas a tarefas e técnicos, e contribuir para a rastreabilidade das operações realizadas.

Como resultado, foi desenvolvido o sistema informático *SmartTool Cabinet*, centrado numa interface móvel para os principais intervenientes no processo de gestão de ferramentas. Através desta interface, os utilizadores podem aceder às funcionalidades adequadas ao

seu perfil, consultar informação relevante sobre ferramentas e tarefas, e executar operações associadas ao levantamento, devolução e controlo do estado das ferramentas. Para permitir a demonstração funcional da aplicação, foi também desenvolvida uma componente de servidor, responsável por centralizar os dados e aplicar as regras de negócio necessárias ao funcionamento do protótipo.

A solução distingue-se por por três aspetos principais. Em primeiro lugar, a aplicação foi desenvolvida em Kotlin Multiplatform (KMP), permitindo a partilha de uma única base de código entre Android e iPhone *Operating System* (iOS). Em segundo lugar, suporta autenticação por email e *password*, assim como autenticação federada por *single sign-on*, facilitando a integração com mecanismos de autenticação já existentes. Por fim, articula a interface móvel com uma *Application Programming Interface* (API), permitindo manter a informação consistente entre os diferentes utilizadores e registar as operações realizadas na aplicação.

O foco deste projeto está na operação e no uso diário da aplicação móvel. Por esse motivo, a criação e o carregamento inicial do inventário base de ferramentas, utilizadores e armários ficam fora do âmbito deste trabalho. Assume-se que estes dados já existem no contexto organizacional em que a solução se insere, sendo os dados usados no projeto apenas valores de teste para demonstrar o funcionamento do sistema.

Todo o código desenvolvido no âmbito deste projeto encontra-se disponível para consulta num repositório público, acessível a partir do seguinte *link*: <https://github.com/lulssam/SmartTool-Cabinet>.

1.2 Trabalho Relacionado

No contexto da automação da gestão de inventário, a implementação de armários inteligentes tem sido explorada para otimizar operações, garantir o acompanhamento contínuo da localização e histórico de cada item, e minimizar falhas associadas a processos manuais.

Na área da saúde, a implementação de armários inteligentes baseados em *Radio-Frequency Identification* (RFID) foi avaliada com o objetivo de gerir a rastreabilidade de material médico de alto custo e de medicamentos [del Carmen León-Araujo et al., 2019]. A utilização desta tecnologia permitiu o registo automático das entradas e saídas de produtos, contribuindo para a eliminação de ruturas de *stock* e para uma redução de 58% do tempo despendido em tarefas de gestão logística. Adicionalmente, foi demonstrado que um controlo de inventário mais seguro pode ser alcançado com a utilização destes sistemas, reduzindo erros associados aos processos humanos manuais e às limitações dos códigos de barras convencionais [Gabriels et al., 2016].

Em ambientes domésticos, foi desenvolvido um protótipo de armário de cozinha inteligente, concebido para monitorizar mercearias através de uma arquitetura que combina dados recolhidos por sensores de peso com leituras de etiquetas RFID [Pal Amutha et al., 2012]. Neste sistema, a análise contínua das variações de peso permite identificar a entrada ou a retirada de produtos, atualizando a base de dados central em conformidade.

Numa vertente focada no controlo de equipamentos e na gestão de espaços, foi demonstrada a viabilidade de usar o microcontrolador *Arduino* em conjunto com módulos *Bluetooth* para integrar mecanismos físicos com aplicações móveis e bases de dados

em tempo real [Hamzah et al., 2021]. Neste tipo de arquitetura, a comunicação com a base de dados é estabelecida pela aplicação móvel do utilizador para gerir itens previamente registados por códigos *Quick Response* (QR), sendo simultaneamente enviados comandos sem fios ao microcontrolador para acionar os atuadores lineares do armário.

Fora do contexto industrial, o conceito de cacifos inteligentes encontra-se amplamente difundido em soluções comerciais de logística, como os cacifos automáticos *Locky* dos Correios de Portugal (CTT) [CTT — Correios de Portugal, 2026]. Nestes sistemas, cada compartimento permanece trancado e só é aberto após a validação da identidade do utilizador, normalmente através de um código único, um *Personal Identification Number* (PIN) ou um código QR, gerado no momento da entrega. O sistema central regista que encomenda se encontra em cada cacifo e a quem se destina, associando cada levantamento a um utilizador específico.

A implementação destes métodos de identificação automática resulta num controlo rigoroso e centralizado, eliminando as discrepâncias de inventário, prevenindo o extravio de materiais e reduzindo significativamente o tempo despendido em tarefas de gestão logística e administrativa. A análise destas soluções mostra a relevância da identificação automática, do registo centralizado e da rastreabilidade na gestão de objetos físicos. No entanto, as abordagens analisadas não respondem diretamente ao contexto específico da manutenção aeronáutica, onde é necessário articular ferramentas, técnicos, tarefas, armários e histórico de utilização. É neste enquadramento que se definem os objetivos do presente trabalho.

1.3 Utilização de IA no contexto deste trabalho

Ao longo do desenvolvimento deste projeto foram utilizadas ferramentas de Inteligência Artificial (IA) como mecanismo de apoio pontual em tarefas específicas. A sua principal utilização ocorreu durante a implementação do servidor desenvolvido em *Ktor*, uma vez que esta tecnologia era inicialmente desconhecida pelos membros do grupo. Neste contexto, a IA foi utilizada para compreender a estrutura do *framework*, esclarecer dúvidas de implementação, obter exemplos de desenvolvimento de determinadas funcionalidades e apoiar a definição da comunicação entre o servidor e a base de dados.

A conceção da base de dados em *MySQL* foi realizada sem recurso a ferramentas de IA, bem como todo o modelo de dados, o desenho do esquema relacional, as entidades, as relações e as decisões de modelação.

Da mesma forma, toda a interface gráfica da aplicação foi idealizada, desenhada e implementada sem recurso a IA, desde a análise das necessidades até à definição dos fluxos de navegação, dos protótipos e da experiência de utilização. O recurso à IA nesta componente limitou-se a sugestões pontuais para pequenas correções da interface ou ao esclarecimento da melhor forma de obter e apresentar dados provenientes da base de dados através do servidor.

Para o desenvolvimento do servidor em *Ktor*, uma tecnologia com que os membros do grupo ainda não tinham trabalhado, assinala-se que a utilização de ferramentas de IA se revelou um apoio importante na aprendizagem e implementação dessa componente, sem substituir as decisões de arquitetura, de modelação ou de desenho da solução.

Neste projeto, a identificação de código desenvolvido pelo grupo através do marcador

`mycode` foi aplicada apenas nas componentes consideradas mais relevantes para demonstrar a autoria da solução.

Foi utilizado nos seguintes elementos:

- DTOs (*Data Transfer Objects*);
- *UI States*;
- Definição das tabelas da base de dados MySQL;
- Endpoints `GET` implementados no servidor `Ktor` .

Por outro lado, optou-se por não utilizar a marcação `mycode`, na maioria das seguintes componentes:

- *ViewModels*;
- *Screens* da interface gráfica;
- Inserção de dados (`INSERT`) na base de dados;
- *Views* da base de dados;
- Restantes componentes do *backend*.

No repositório do projeto encontram-se identificadas as *prompts* mais importantes que foram utilizadas durante o seu desenvolvimento.

1.4 Organização do Documento

O presente relatório está organizado em seis capítulos, incluindo o presente capítulo introdutório.

O Capítulo 2 apresenta os requisitos e o modelo funcional e do sistema informático, identificando os perfis de utilizador, os requisitos funcionais e não funcionais e os principais casos de utilização. De seguida, o Capítulo 4 apresenta a arquitetura e a conceção do sistema informático, começando pela arquitetura geral e seguindo depois para as principais opções adotadas ao nível da persistência dos dados, da comunicação entre a aplicação móvel e a componente de servidor, do desenvolvimento multiplataforma, da organização interna da aplicação móvel, da autenticação e da abordagem de desenvolvimento. Depois, o Capítulo 4 descreve a Implementação, detalhando o modelo de dados, a `API REST` e a aplicação móvel multiplataforma. Depois o Capítulo 5 apresenta a validação e os testes realizados sobre o protótipo, com base na simulação dos principais fluxos de utilização da aplicação. São considerados mecanismos de autenticação e acesso, a atribuição de tarefas, o fluxo de atribuição e requisição de ferramentas, e ainda a devolução e o reporte de anomalias. Por fim, o Capítulo 6 reúne as *Conclusões* do trabalho desenvolvido e identifica as linhas de *Trabalho Futuro* que permitirão evoluir o sistema.

Capítulo 2

Requisitos e modelo funcional

Neste capítulo é apresentado o modelo funcional do sistema informático desenvolvido. Começa-se por identificar os perfis de utilizador previstos – Técnico, Gestor e *Back-Office* – e as principais responsabilidades e interações associadas a cada um. De seguida, são apresentados os requisitos funcionais e não funcionais que orientaram a conceção da solução, bem como os casos de utilização que descrevem a interação dos utilizadores com a aplicação móvel.

2.1 Perfis de Utilizador

O sistema informático desenvolvido considera três perfis principais de utilizador: Técnico, Gestor e Back-Office. Cada perfil tem responsabilidades distintas no processo de gestão de ferramentas e, por esse motivo, tem acesso a funcionalidades diferentes na aplicação móvel.

Técnico

O perfil de Técnico corresponde ao utilizador responsável pela execução das tarefas de manutenção. Através da aplicação móvel, este utilizador pode consultar a informação associada às suas tarefas, requisitar ferramentas e, no momento da devolução, indicar se alguma ferramenta se encontra em mau estado.

Gestor

O perfil de Gestor está associado à coordenação operacional das tarefas e das ferramentas disponíveis. Este utilizador pode atribuir tarefas e ferramentas aos técnicos, consultar ferramentas em falta e os respetivos detentores, verificar o estado das ferramentas e dos armários, visualizar alertas ativos, analisar o histórico do sistema e gerir o catálogo global de ferramentas.

Back-Office

O perfil de *Back-Office* está associado à gestão administrativa dos utilizadores e dos respetivos acessos. Este utilizador pode gerir funcionários, controlar os níveis de acesso à aplicação e definir turnos de trabalho.

Esta distinção entre perfis permite organizar as funcionalidades da aplicação de acordo com as responsabilidades de cada utilizador, garantindo que cada perfil acede apenas à informação e às operações necessárias ao desempenho das suas funções.

2.2 Requisitos

A elaboração de requisitos é uma fase fundamental para garantir que o projeto responde com eficácia às rigorosas necessidades de segurança e rastreabilidade da manutenção aeronáutica. O resultado desta análise está estruturado em duas grandes categorias: os Requisitos Funcionais, que definem as ações e regras de negócio que o sistema deve executar, e os Requisitos Não Funcionais, que estabelecem as restrições técnicas, os atributos de qualidade e os padrões de usabilidade.

2.2.1 Requisitos Funcionais

Os requisitos funcionais foram definidos tendo em conta os três perfis de utilizador apresentados na secção anterior(2.1), de modo a garantir que cada perfil tem acesso às funcionalidades adequadas às suas responsabilidades. Os requisitos não funcionais complementam esta definição, estabelecendo condições relacionadas com a utilização, as plataformas suportadas, a acessibilidade e a qualidade da aplicação.

Requisitos do sistema centram-se na segurança e no controlo operacional da aplicação de gestão de ferramentas. A Tabela 2.1 apresenta os requisitos funcionais definidos para garantir o acesso seguro à aplicação e a rastreabilidade das operações efetuadas pelos utilizadores.

Tabela 2.1: Requisitos Funcionais do Sistema

Requisito	Descrição	Tipo
RF1	Apenas funcionários registados pelo Back Office podem ter acesso à aplicação	Invisível
RF2	Guardar histórico de 30 dias das ferramentas requisitadas e devoluções	Invisível

O requisito RF1 garante que apenas funcionários autorizados podem utilizar a aplicação, enquanto o RF2 assegura a retenção do histórico de requisições durante 30 dias, permitindo a rastreabilidade das operações.

Requisitos de gestão de funcionários Os requisitos de gestão de funcionários definem as funcionalidades associadas ao perfil de *Back-Office*, responsável pela gestão administrativa dos utilizadores e dos respetivos acessos. A Tabela 2.2 apresenta os requisitos funcionais definidos para a gestão de funcionários, níveis de acesso e turnos de trabalho.

Tabela 2.2: Requisitos de Gestão de Funcionários

Requisito	Descrição	Tipo
RF3	Permitir ao <i>Back-Office</i> adicionar e remover funcionários	Evidente
RF4	Permitir ao <i>Back-Office</i> controlar os níveis de acesso à aplicação	Evidente
RF5	Permitir ao <i>Back-Office</i> definir turnos de horário	Evidente

Os requisitos RF3, RF4 e RF5 garantem que o perfil de *Back-Office* dispõe das funcionalidades necessárias para gerir os utilizadores da aplicação, controlar os respetivos níveis de acesso e definir os turnos de trabalho associados.

Requisitos da interface gráfica têm como objetivo fornecer aos Técnicos e Gestores uma visualização clara e atualizada da informação necessária ao desempenho das suas funções. A Tabela 2.3 apresenta os requisitos funcionais definidos para a interface da aplicação.

Tabela 2.3: Requisitos Funcionais da Interface Gráfica

Requisito	Descrição	Tipo
RF6	Mostrar o histórico de 30 dias das ferramentas requisitadas ao Gestor e Técnico	Evidente
RF7	Mostrar ferramentas disponíveis ao Gestor e Técnico	Evidente
RF8	Mostrar ao Gestor o estado dos armários	Evidente
RF9	Mostrar ao Gestor alertas ativos	Evidente
RF10	Mostrar ao Técnico as tarefas que lhe estão associadas	Evidente
RF11	Mostrar ao Gestor ferramentas em falta e respetivos detentores	Evidente

Os requisitos funcionais descritos asseguram que cada perfil de utilizador acede apenas à informação pertinente para as suas funções. Em particular, o RF6 e o RF7 permitem a ambos os perfis (Gestor e Técnico) consultar o histórico de requisições dos últimos 30 dias e as ferramentas disponíveis, respetivamente. Para funções de supervisão, os requisitos RF8, RF9 e RF11 facultam ao Gestor o acesso ao estado dos armários, aos alertas ativos e à monitorização de ferramentas em falta. Em contrapartida, o RF10 garante que o Técnico consegue visualizar as tarefas que lhe foram atribuídas.

Requisitos de tarefas e ferramentas definem as operações que permitem ao Gestor organizar o trabalho dos técnicos e associar ferramentas às tarefas de manutenção. A Tabela 2.4 apresenta os requisitos funcionais definidos para a atribuição de tarefas e para a gestão das ferramentas associadas.

Tabela 2.4: Requisitos Funcionais de Tarefas e Ferramentas

Requisito	Descrição	Tipo
RF12	Permitir ao Gestor atribuir tarefas aos técnicos	Evidente
RF13	Permitir ao Gestor associar ferramentas às tarefas atribuídas	Evidente
RF14	Permitir ao Gestor gerir o catálogo global de ferramentas	Evidente

Os requisitos RF12, RF13 e RF14 permitem relacionar tarefas, técnicos e ferramentas, assegurando que o Gestor consegue organizar o trabalho previsto e consultar ou atualizar a informação necessária à gestão das ferramentas.

Requisitos de requisição e devolução definem as operações realizadas pelo Técnico durante a utilização diária da aplicação. A Tabela 2.5 apresenta os requisitos funcionais definidos para a requisição, devolução e reporte de anomalias em ferramentas.

Tabela 2.5: Requisitos Funcionais de Requisição e Devolução

Requisito	Descrição	Tipo
RF15	Permitir ao Técnico requisitar ferramentas associadas às suas tarefas	Evidente
RF16	Permitir ao Técnico devolver ferramentas requisitadas	Evidente
RF17	Permitir ao Técnico reportar anomalias no momento da devolução, nomeadamente quando uma ferramenta se encontra em mau estado	Evidente

Os requisitos RF15, RF16 e RF17 asseguram o registo das principais operações efetuadas pelo Técnico, permitindo acompanhar o levantamento e a devolução das ferramentas, bem como eventuais alterações ao seu estado.

Requisitos de notificação foram definidos para assegurar uma resposta rápida perante situações críticas. A Tabela 2.6 apresenta o requisito funcional responsável pela notificação automática do Gestor.

Tabela 2.6: Requisitos Funcionais de Notificação

Requisito	Descrição	Tipo
RF18	Enviar ao Gestor uma notificação caso exista um alerta de grau de importância elevado	Evidente

O requisito RF18 garante que o Gestor tem acesso aos alertas ativos, permitindo uma intervenção mais rápida e reduzindo o impacto de eventuais anomalias.

Requisitos de autenticação garantem um acesso seguro à aplicação e compatível com as políticas de segurança da organização. A Tabela 2.7 apresenta os métodos de autenticação suportados.

Tabela 2.7: Requisitos Funcionais de Autenticação

Requisito	Descrição	Tipo
RF19	O sistema permite autenticação de utilizadores com autenticação federada	Evidente
RF20	O sistema permite autenticação de utilizadores com email e palavra-passe	Evidente

Os requisitos RF19 e RF20 permitem que a aplicação suporte autenticação federada e autenticação por email e palavra-passe, garantindo flexibilidade e compatibilidade com os mecanismos de autenticação da organização.

2.2.2 Requisitos Não Funcionais

Requisitos estéticos asseguram que a aplicação mantém uma identidade visual consistente com a identidade visual definida para o protótipo. A Tabela 2.8 apresenta os requisitos não funcionais considerados.

Tabela 2.8: Requisitos Não Funcionais Estéticos

Requisito	Descrição	Tipo
RnF1	A paleta de cores da aplicação deve seguir a identidade visual definida para o protótipo	Desejável
RnF2	Garantir que os ícones seguem o mesmo estilo gráfico	Desejável

Os requisitos RnF1 e RnF2 promovem uma interface visual coerente com a identidade definida para o protótipo, reforçando a consistência gráfica e o reconhecimento da aplicação pelos utilizadores.

Requisitos de facilidade de utilização procuram reduzir a curva de aprendizagem e aumentar a eficiência dos utilizadores. A Tabela 2.9 apresenta o requisito definido.

Tabela 2.9: Requisitos Não Funcionais da Facilidade de Utilização

Requisito	Descrição	Tipo
RnF3	Navegação de menus simples e intuitiva	Desejável

O requisito RnF3 privilegia uma navegação simples e intuitiva, reduzindo erros de utilização e facilitando a execução das tarefas diárias.

Requisitos de plataforma definem os sistemas operativos suportados pela aplicação. A Tabela 2.10 apresenta os requisitos considerados.

Tabela 2.10: Requisitos Não Funcionais de Plataformas

Requisito	Descrição	Tipo
RnF4	Android	Obrigatório
RnF5	iOS	Obrigatório
RnF6	Desktop	Desejável

Os requisitos RnF4 e RnF5 garantem o funcionamento da aplicação nos dispositivos móveis utilizados pelos colaboradores, enquanto o RnF6 prevê suporte para Desktop como funcionalidade complementar.

Requisitos de interação pessoa-máquina procuram assegurar uma experiência de utilização fluida através de tempos de resposta reduzidos. A Tabela 2.11 apresenta o requisito definido.

Tabela 2.11: Requisitos Não Funcionais de Interação Pessoa-Máquina

Requisito	Descrição	Tipo
RnF7	Resposta rápida do sistema	Desejável

O requisito RnF7 estabelece que o sistema deve responder rapidamente às ações do utilizador, contribuindo para uma maior eficiência durante a utilização da aplicação.

Requisitos de acessibilidade visam garantir que a aplicação pode ser utilizada por diferentes perfis de utilizadores. A Tabela 2.12 apresenta o requisito considerado.

Tabela 2.12: Requisitos Não Funcionais de Acessibilidade

Requisito	Descrição	Tipo
RnF8	Interface legível e acessível para os diferentes perfis de utilizador	Desejável

O requisito RnF8 complementa a facilidade de utilização, promovendo uma interface legível e acessível e, uma navegação intuitiva que facilita a interação dos diferentes perfis de utilizadores com a aplicação.

2.2.3 Casos de Utilização

Os casos de utilização têm como principal objetivo descrever as interações entre os utilizadores e o sistema, detalhando de forma visual as funcionalidades e operações disponíveis para cada perfil. Nesta secção, apresentam-se os diagramas correspondentes e as respectivas descrições, estruturados de acordo com os três perfis principais da aplicação: *Back Office*, Gestor e Técnico.

Estes casos de utilização permitem relacionar os requisitos definidos na secção anterior com as funcionalidades disponibilizadas pela aplicação móvel, evidenciando a separação entre as operações administrativas, as operações de coordenação e as operações realizadas durante a utilização diária das ferramentas.

Back Office

A Figura 2.1 apresenta os Casos de Utilização referentes ao perfil de *Back Office*.

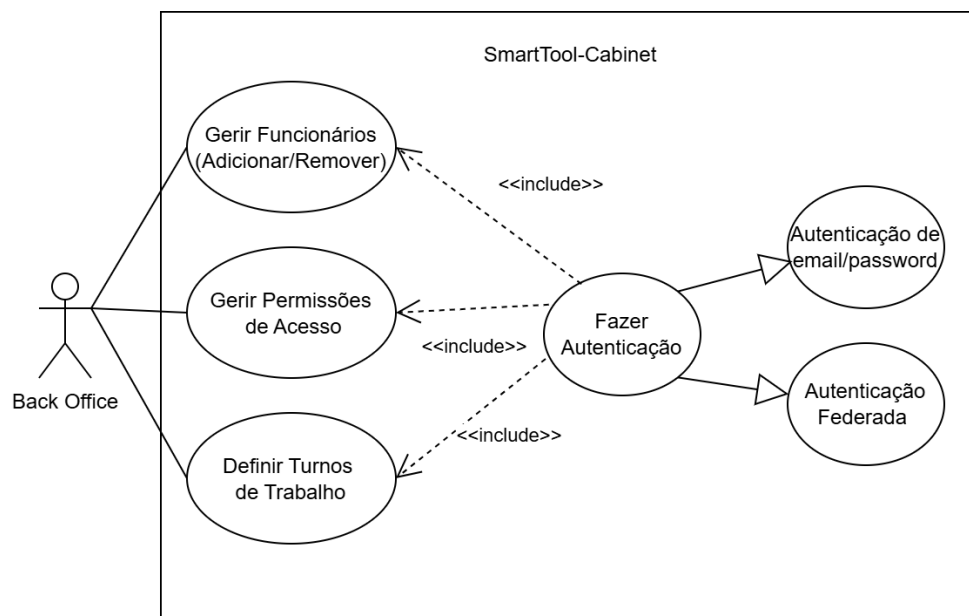


Figura 2.1: Diagrama de casos de utilização do *Back Office*

Conforme ilustrado no diagrama apresentado na Figura 2.1, numa primeira fase, é exigido que o utilizador efetue a sua autenticação no sistema, a qual pode ser realizada através de credenciais tradicionais (email e palavra-passe) ou via autenticação federada. Uma vez autenticado, o utilizador dispõe de permissões para gerir funcionários, nomeadamente a sua adição e remoção, controlar os níveis de acesso à aplicação e definir os respetivos turnos de trabalho.

Gestor

A Figura 2.2 apresenta os Casos de Utilização referentes ao perfil de Gestor.

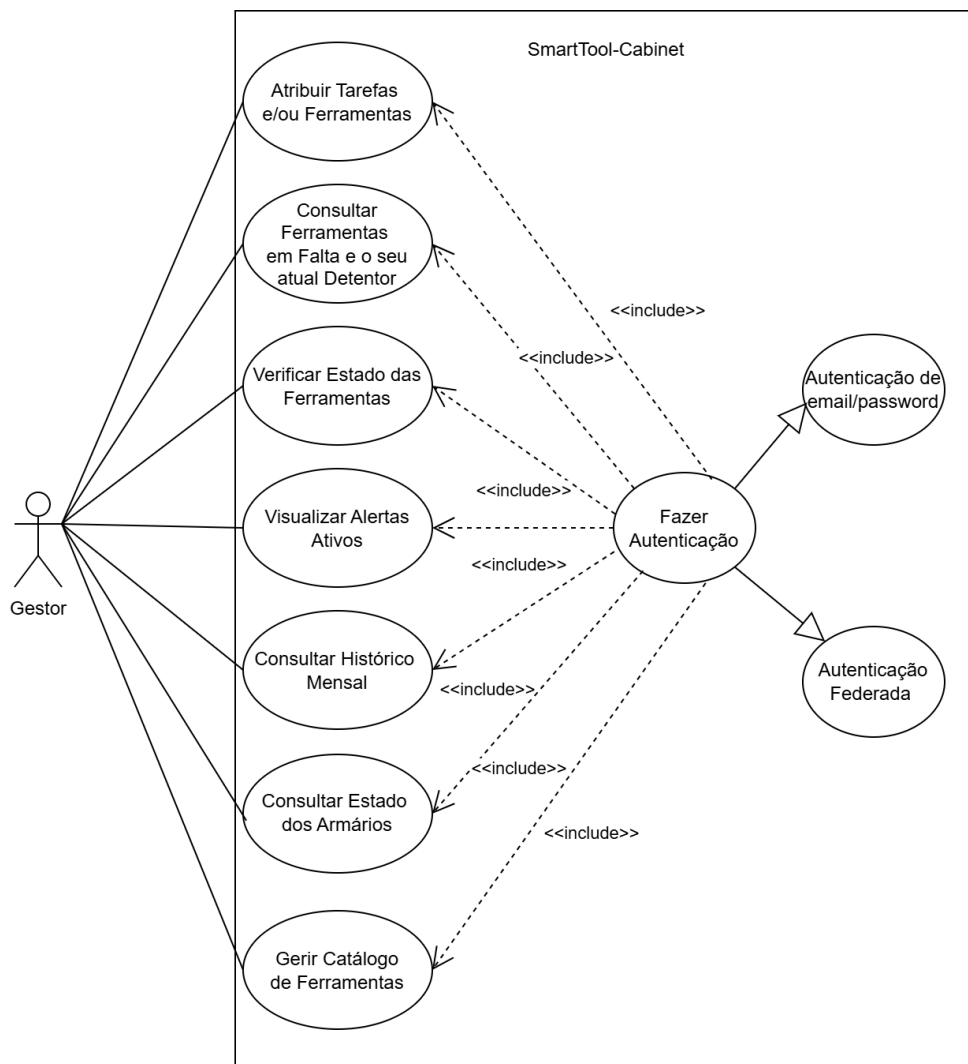


Figura 2.2: Diagrama Caso de Utilização Gestor

Conforme ilustrado no diagrama apresentado na Figura 2.2, numa primeira fase, é exigido que o utilizador efetue a sua autenticação no sistema, a qual pode ser realizada através de credenciais tradicionais (email e palavra-passe) ou via autenticação federada. Uma vez autenticado, o utilizador dispõe de permissões para atribuir tarefas e ferramentas, consultar ferramentas em falta e os seus respetivos detentores, verificar o estado das mesmas e visualizar os alertas ativos. Adicionalmente, o Gestor está habilitado a analisar o histórico do sistema, monitorizar o estado operacional dos armários e gerir o catálogo global de ferramentas.

Técnico

A Figura 2.3 apresenta os Casos de Utilização referentes ao perfil de Técnico.

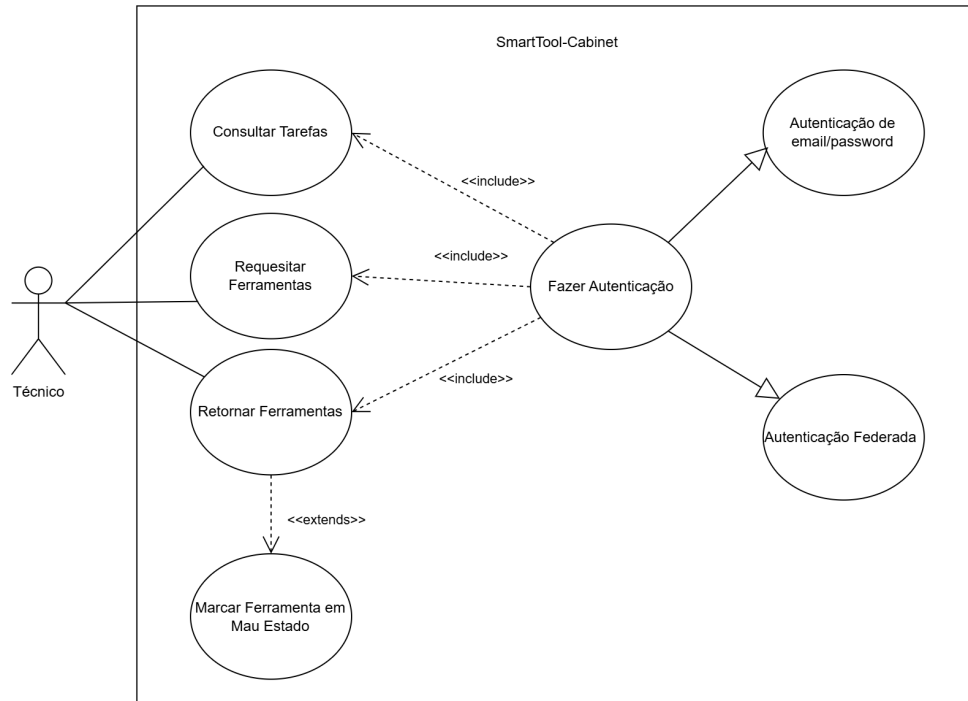


Figura 2.3: Diagrama Caso de Utilização Técnico

Conforme ilustrado no diagrama apresentado na Figura 2.3, numa primeira fase, é exigido que o utilizador efetue a sua autenticação no sistema, a qual pode ser realizada através de credenciais tradicionais (email e palavra-passe) ou via autenticação federada. Uma vez autenticado, o utilizador dispõe de permissões para consultar tarefas, requisitar ferramentas e, no momento da sua devolução, marcá-las como em mau estado, caso isso se verifique.

Capítulo 3

Arquitetura do sistema informático

Neste capítulo é apresentada a arquitetura geral do sistema informático desenvolvido e as principais opções de conceção adotadas no seu desenvolvimento. Começa-se por descrever a organização global do sistema, mostrando a relação entre a aplicação móvel, a componente de servidor e a base de dados. De seguida, são apresentadas as opções associadas à persistência dos dados, à comunicação entre a aplicação móvel e o servidor, ao desenvolvimento multiplataforma, à organização interna da aplicação móvel e aos mecanismos de autenticação.

3.1 Persistência e rastreabilidade de dados

Para guardar e gerir a informação do sistema, foi escolhida uma base de dados relacional usando o MySQL [Oracle Corporation, 2026]. Neste modelo, os dados são organizados em tabelas (relações), cada uma representando uma entidade do domínio do problema, como funcionários, cargos, ferramentas, armários, tarefas e requisições.

Esta opção foi considerada adequada ao sistema desenvolvido, uma vez que existem várias relações entre entidades. Por exemplo, um técnico pode ter tarefas atribuídas, uma tarefa pode estar associada a ferramentas e uma requisição deve permitir identificar que ferramenta foi levantada, por que técnico e em que momento. Desta forma, a utilização de uma base de dados relacional contribui para manter a consistência dos dados e para apoiar a rastreabilidade das operações realizadas.

No que respeita à eliminação de dados, decidiu-se aplicar a técnica de *soft delete*. Em vez de os registos serem completamente apagados com o comando `DELETE` (o que pode violar regras de integridade referencial ou obrigar a apagar dados em cascata), estes são marcados como inativos através de uma coluna de estado. Esta decisão permite preservar o histórico de todas as ações associadas a esse registo, evitando a perda de informação relevante para a análise posterior das operações realizadas no sistema.

Para a comunicação entre o servidor e a base de dados, foi adotada a tecnologia *Java Database Connectivity* (JDBC). Para garantir que a informação nunca fica num estado inválido quando é preciso alterar várias tabelas ao mesmo tempo, estes comandos são agrupados em transações. Se ocorrer algum erro durante o processo, desfazem-se estas operações, garantindo que nada é gravado pela metade e que a base de dados se mantém consistente [Trigo et al., 2026].

3.2 Comunicação entre aplicação móvel e componente de servidor

O sistema funciona com um modelo cliente-servidor. Esta opção permite separar a aplicação móvel, utilizada pelos diferentes perfis de utilizador, da componente de servidor, responsável por centralizar os dados e aplicar as regras de negócio [Fielding, 2000]. Esta separação é útil porque permite que cada parte seja desenvolvida e melhorada de forma independente.

Para organizar a forma como a aplicação e a componente de servidor comunicam, foi usado o modelo **REST**. Uma das principais regras deste modelo, é que funciona de forma *stateless* (sem estado), o servidor não precisa de ter um histórico de comunicações, uma vez que cada pedido feito pelo cliente já leva toda a informação necessária para o servidor perceber o que tem de fazer [Fielding, 2000].

A troca de mensagens é feita através do protocolo *Hypertext Transfer Protocol* (HTTP), usando os seguintes métodos:

- **GET**: usado apenas para pedir e ler dados.
- **POST**: usado para enviar novos dados para o servidor.
- **PATCH**: usado para fazer alterações a dados que já existem.

A ausência deliberada do método HTTP **DELETE**, completando o ciclo CRUD apenas de forma parcial, constitui uma decisão arquitetural relacionada com a necessidade de preservar a informação relevante para a rastreabilidade das operações. Numa atividade altamente regulada como a manutenção aeronáutica, a remoção física de registos destruiria o histórico. Para isso, todas as operações de exclusão são tratadas de forma lógica (*soft delete*) através do método **PATCH**, desativando utilizadores mas retendo os seus dados para total rastreabilidade.

Depois de receber um pedido, o servidor devolve códigos de estado HTTP. Estes códigos servem para avisar a aplicação de forma clara, sobre o resultado da operação realizada: se o pedido teve sucesso, se o utilizador não tem autorização ou enviou alguma coisa mal ou se ocorreu um erro interno.

A comunicação entre a aplicação móvel e a componente de servidor é feita em formato *JavaScript Object Notation* (JSON). Este formato foi adotado por ser simples de processar na aplicação e adequado à troca de dados estruturados entre o cliente e a API **REST**.

Em termos práticos de programação, foi escolhida a ferramenta **Ktor** [JetBrains, 2026c]. Esta ferramenta foi usada para implementar a API **REST** da componente de servidor, responsável por receber os pedidos da aplicação móvel e encaminhá-los para as operações necessárias sobre a base de dados.

3.3 Desenvolvimento multiplataforma da aplicação móvel

Para o desenvolvimento da aplicação móvel, optou-se pela utilização da tecnologia **Kotlin Multiplatform** (KMP) [JetBrains, 2026b], que permite a partilhar uma única base de

código entre aplicações para os sistemas operativos Android e iOS. A escolha desta tecnologia foi fortemente impulsionada pela familiaridade prévia com a linguagem `Kotlin`, adquirida no âmbito da unidade curricular Desenvolvimento de Aplicações Móveis.

Adicionalmente, esta abordagem destaca-se de outras alternativas multiplataforma, como o `Flutter` ou o `React Native`, por oferecer a flexibilidade de partilhar a lógica de negócio, garantindo, em simultâneo, a integração com funcionalidades nativas de cada plataforma. Esta opção foi considerada adequada ao projeto, uma vez que permitiu reduzir a duplicação de código e facilitar a manutenção da aplicação, sem comprometer o objetivo de disponibilizar a solução nos dois principais sistemas operativos móveis.

A interface gráfica da aplicação foi construída com recurso ao `Compose Multiplatform` [JetBrains, 2026a], uma ferramenta concebida para a criação e partilha de componentes visuais entre várias plataformas. A adoção desta ferramenta permitiu manter uma abordagem coerente no desenvolvimento da interface, tirando partido da base de código comum definida para a aplicação.

Para gerir os comportamentos que precisam de variar consoante o sistema operativo sem afetar o código partilhado, foi adotado o mecanismo `expect/actual`. Este mecanismo permite a declaração de uma expectativa (`expect`) no código comum, cuja implementação concreta (`actual`) é fornecida nos módulos nativos de cada plataforma. Neste sistema, este padrão é usado para resolver o endereço de comunicação com o servidor, visto que o endereço varia dependendo se a aplicação está a correr num emulador Android, num simulador iOS ou num dispositivo físico.

3.4 Organização interna da aplicação móvel

Para organizar internamente a aplicação móvel, foram adotados padrões e mecanismos que permitem separar responsabilidades, tornar o estado da interface mais previsível e facilitar a manutenção do código

O padrão MVVM (Model-View-ViewModel) constitui a base da interface gráfica da aplicação. Neste modelo, a parte visual, designada por *View*, funciona apenas como uma observadora, delegando a gestão dos eventos e da lógica de apresentação para o *ViewModel* [Android Developers, 2024]. A adoção do MVVM, em detrimento de outros padrões como o *Model-View-Controller* (MVC), justifica-se pelas vantagens desta arquitetura na separação de responsabilidades. No MVC, o controlador tende a ficar fortemente acoplado à *View*, enquanto no MVVM o *ViewModel* permanece independente da interface, facilitando a realização de testes, a manutenção e a organização do código [GeeksforGeeks, 2025]. Esta escolha foi ainda reforçada pela familiaridade adquirida com este padrão durante a unidade curricular Desenvolvimento de Aplicações Móveis.

O fluxo de dados unidirecional foi de dados unidirecional foi adotado como complemento da arquitetura MVVM, garantindo que a informação circula sempre num único sentido. A Figura 3.1 ilustra a interação entre os diferentes componentes da arquitetura e o respetivo fluxo de dados.

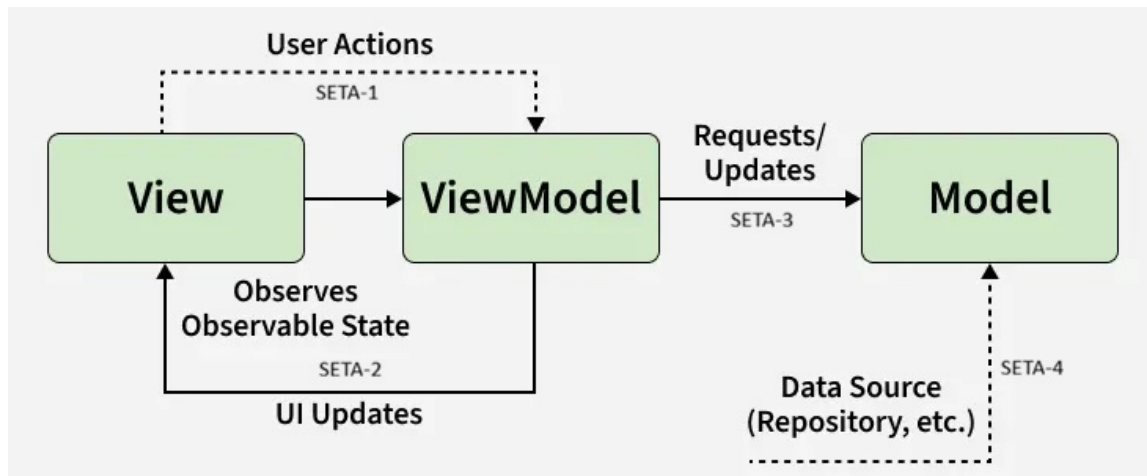


Figura 3.1: Estrutura do padrão MVVM e fluxo de dados unidirecional. Fonte: [GeeksforGeeks, 2025].

Como se observa na Figura 3.1, as ações do utilizador são comunicadas da *View* para o *ViewModel* (SETA-1). Após o processamento desses eventos, o *ViewModel* atualiza o estado da interface, que é observado pela *View* (SETA-2). Sempre que necessita de obter ou atualizar informação, o *ViewModel* comunica com o *Model* (SETA-3), que, por sua vez, interage com a respetiva fonte de dados (SETA-4). Esta separação clara de responsabilidades torna o estado da aplicação mais previsível, simplifica o processo de depuração e facilita a manutenção do código [Android Developers, 2024].

O Estado imutável foi utilizado para representar o estado de cada ecrã através de classes de dados estritamente imutáveis. Sempre que é necessário alterar o estado, não é efetuada uma modificação direta; é criada uma nova instância que reflete as alterações pretendidas. Esta abordagem reduz a possibilidade de inconsistências e facilita o acompanhamento da evolução do estado da aplicação.

Corrotinas permitem executar operações assíncronas, como pedidos ao servidor, sem bloquear a interface do utilizador. A sua utilização contribui para uma aplicação mais fluida, assegurando que tarefas potencialmente demoradas são executadas em segundo plano de forma eficiente.

Navegação *type-safe* foi adotada para garantir que a navegação entre ecrãs é validada em tempo de compilação. Em vez de identificar os destinos através de cadeias de texto, cada rota é representada por um objeto tipado e serializável. Desta forma, eliminam-se erros associados a nomes de rotas incorretos e aumenta-se a robustez da aplicação.

Injeção de dependências (DI) foi utilizada para fornecer a cada componente apenas as dependências de que necessita, sem que este seja responsável pela sua criação. Esta abordagem reduz o acoplamento entre classes, melhora a organização do código e facilita a substituição de implementações durante o desenvolvimento e os testes.

Gestão do estado de sessão foi organizada de forma semelhante a uma máquina de estados, permitindo determinar automaticamente a situação do utilizador durante o arranque da aplicação. Consoante exista uma sessão válida, uma sessão inexistente ou um estado de carregamento, o utilizador é encaminhado para o ecrã apropriado.

Padrão de repositório foi adotado para isolar o acesso aos dados da restante lógica da aplicação através de uma camada de abstração. O código partilhado comunica apenas com interfaces, sendo as implementações concretas fornecidas por cada plataforma (Android ou iOS). Além disso, os resultados das operações são representados através de *sealed classes* [Kotlin Documentation, 2024], permitindo modelar explicitamente cenários de sucesso e erro e garantindo que todos os casos possíveis são tratados pela aplicação.

3.4.1 Autenticação e controlo de acesso

A segurança de um sistema exige uma distinção clara entre autenticação e autorização: a autenticação confirma a identidade do utilizador, enquanto a autorização define, depois disso, a que ações e recursos essa identidade tem acesso. No contexto deste projeto, esta distinção é particularmente importante, uma vez que a aplicação disponibiliza funcionalidades diferentes aos perfis de Técnico, Gestor e *Back-Office*.

Para a validação da identidade dos utilizadores, optou-se pela utilização do o Firebase Authentication [Google, 2026]. Esta plataforma de Identidade como Serviço (*Identity-as-a-Service*), valida apenas a identidade do utilizador, mantendo a autorização, isto é, cargos, perfis e permissões, sob controlo do servidor do próprio sistema.

A verificação da identidade pode ser feita de forma tradicional, através de *email* e palavra-passe, ou delegada num fornecedor externo, através de autenticação federada e do mecanismo de *Single Sign-On (SSO)*, que permite a um utilizador aceder a várias aplicações com as mesmas credenciais. Esta opção foi considerada adequada ao projeto porque permite suportar o acesso por credenciais próprias da aplicação e, simultaneamente, preparar a solução para integração futura com mecanismos de autenticação já existentes na organização.

Esta comunicação assenta em dois protocolos: o *Open Authorization (OAuth)* 2.0, que permite obter acesso limitado aos dados do utilizador sem expor a sua palavra-passe, e o OpenID Connect, que se constrói sobre o OAuth 2.0 para acrescentar a verificação de identidade.

Depois de validada a identidade do utilizador, a componente de servidor é responsável por associar esse utilizador ao respetivo perfil funcional. Assim, o sistema consegue determinar se o utilizador deve aceder às funcionalidades de Técnico, Gestor ou *Back-Office*, garantindo que cada perfil apenas utiliza as operações previstas para as suas responsabilidades.

Por fim, o tratamento das respostas destes serviços segue o mesmo mecanismo de *sealed classes* usado no restante sistema (secção 3.4, Padrões de Arquitetura de Software), garantindo que sucessos e falhas de autenticação são tratados de forma explícita e sem falhas imprevistas.

3.5 Abordagem de desenvolvimento

O objetivo central e o foco no *front-end* refletem-se na abordagem adotada para a concepção do *SmartTool Cabinet*. Desde o primeiro momento, ficou estabelecido que o principal objetivo deste projeto consistia na análise detalhada de um problema logístico real e na criação, de raiz, de uma *User Interface* (UI) capaz de o solucionar. Por esse motivo, o maior esforço de desenvolvimento incidiu sobre o *front-end* e a *User Experience* (UX).

Esta opção é coerente com o âmbito do projeto, uma vez que o trabalho se centrou na aplicação móvel e na forma como os diferentes perfis de utilizador interagem com o sistema. Assim, a interface não foi tratada apenas como uma camada visual, mas como o principal meio de operacionalizar os requisitos definidos para Técnicos, Gestores e *Back-Office*.

A compreensão do contexto operacional constituiu a primeira etapa do desenvolvimento. Para garantir que nenhuma necessidade fosse descurada, o trabalho foi realizado em estreita colaboração com os orientadores, que mantiveram uma ligação contínua com os responsáveis da organização considerada no projeto. Esta comunicação iterativa permitiu compreender as exigências do contexto operacional e traduzi-las em requisitos concretos para o desenho da interface.

A prototipagem e o desenho da interface foram realizados com base nos requisitos identificados durante a fase de análise. A utilização do *Figma* permitiu desenvolver uma linguagem visual consistente e uma arquitetura de navegação adaptada aos três perfis de utilizador: Gestor, Técnico e *Back-Office*. Nesta fase foi dedicado um esforço significativo à definição de fluxos de navegação intuitivos, cartões de informação legíveis e uma interface ergonomicamente adequada ao ambiente de trabalho onde a aplicação será utilizada.

Depois de definida a interface, o desenvolvimento avançou de forma incremental, permitindo implementar e testar progressivamente os principais fluxos da aplicação. Esta abordagem facilitou a validação contínua das funcionalidades desenvolvidas e permitiu ajustar a solução à medida que os requisitos eram clarificados.

O *Backend* como infraestrutura de suporte surge neste projeto como uma componente necessária para permitir a demonstração funcional da aplicação móvel. Embora o foco principal estivesse no *front-end* e na experiência de utilização, o desenvolvimento desta componente permitiu centralizar os dados, disponibilizar a API REST e aplicar as regras de negócio necessárias ao funcionamento do protótipo. Desta forma, foi possível testar fluxos como a autenticação, a consulta de tarefas, a requisição e devolução de ferramentas e o reporte de anomalias.

Capítulo 4

Implementação

Este capítulo descreve a implementação do sistema informático desenvolvido. A aplicação móvel multiplataforma constitui a componente central do projeto, desenvolvida em **Kotlin Multiplatform** para os sistemas Android e iOS. Para suportar o seu funcionamento, foi também desenvolvida uma componente de servidor em **Ktor**, ligada a uma base de dados **MySQL**, que disponibiliza os dados através de uma **API REST**. O capítulo começa por apresentar o modelo de dados, identificando as principais entidades usadas para representar funcionários, cargos, ferramentas, armários, tarefas e requisições (Secção 4.1). De seguida, descreve-se a **API REST**, responsável pela comunicação entre a aplicação móvel e a componente de servidor (Secção 4.2). Por fim, apresenta-se a aplicação móvel multiplataforma, incluindo a sua estrutura, a organização dos ecrãs, os mecanismos de autenticação e a comunicação com a **API** (Secção 4.3).

4.1 Modelo de Dados

A persistência dos dados do sistema assenta numa base de dados **MySQL**. Uma vez que o foco do projeto está na aplicação móvel multiplataforma, o modelo de dados foi mantido propositadamente simples, contemplando apenas as entidades realmente necessárias para suportar as funcionalidades descritas no capítulo 2: gestão de funcionários e respetivos cargos, inventário de ferramentas, armários, requisições e tarefas.

A Figura D.1 apresenta o Modelo Entidade-Associação da base de dados desenvolvida. Nesta representação são evidenciadas apenas as entidades e as relações existentes entre elas, permitindo compreender a organização conceptual da informação.

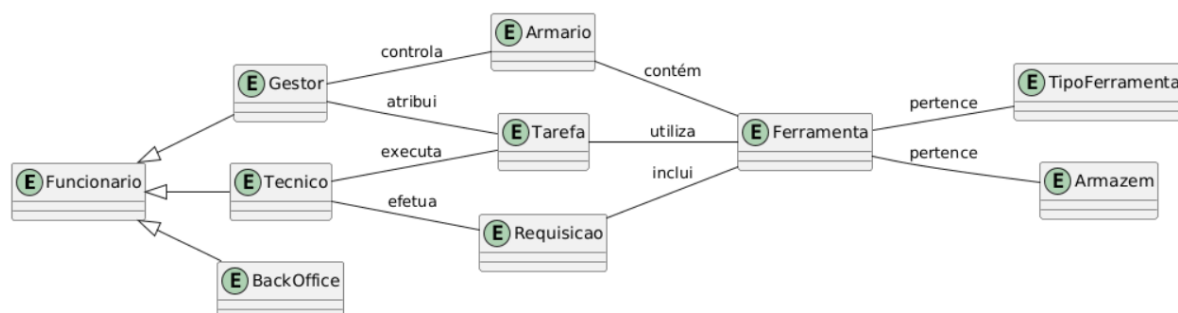


Figura 4.1: Modelo Entidade-Associação da base de dados do sistema.

As entidades do modelo representam os principais conceitos necessários ao funcionamento da aplicação. A entidade **funcionario** representa os dados comuns a todos os utilizadores, como o nome, o *email*, o turno e a *flag* ativo. As entidades **gestor**, **tecnico** e **backoffice** correspondem às especializações de **funcionario**, existindo uma especialização para cada cargo e referenciando a entidade **funcionario** através da chave estrangeira *id_func*.

O **armazem** representa o local onde são guardadas as ferramentas que não se encontram em utilização nas oficinas. O **armario** representa um armário de uma oficina, armazenando informação como a capacidade, o estado e a *flag* trancado. O **tipo_ferramenta** constitui o catálogo dos diferentes tipos de ferramenta, contendo informação como o código, o nome, a categoria e a descrição. Por sua vez, a entidade **ferramenta** representa cada exemplar físico de uma ferramenta, armazenando o seu estado, disponibilidade e localização, que pode corresponder a um armário ou a um armazém.

A entidade **requisicao** representa o levantamento de ferramentas efetuado por um técnico, registando as datas de requisição e de devolução. Por fim, a entidade **tarefa** representa uma tarefa atribuída por um gestor a um técnico, incluindo o respetivo estado e data.

As relações entre as entidades definem a forma como os diferentes elementos do sistema interagem. Um funcionário pode desempenhar um dos cargos existentes através da especialização apresentada no modelo devendo a exclusividade do cargo ser assegurada pelas regras da aplicação ou por restrições adicionais na base de dados. Os gestores controlam armários e atribuem tarefas aos técnicos, enquanto os técnicos executam essas tarefas e efetuam requisições de ferramentas. Cada requisição fica associada à ferramenta ou às ferramentas requisitadas, e cada tarefa pode utilizar um conjunto específico de ferramentas durante a sua execução. Adicionalmente, cada ferramenta pertence a um determinado tipo de ferramenta e encontra-se localizada num armário ou num armazém, quando não se encontra requisitada.

Desta modelação, destacam-se quatro decisões com impacto direto no funcionamento, na manutenção e na rastreabilidade:

A herança por especialização faz com que o cargo de um funcionário não seja guardado numa coluna de tipo, sendo representado pela existência de um registo nas tabelas respetivas, todas referenciando **funcionario**. Esta abordagem permite associar a cada cargo as suas próprias relações, por exemplo, um armário tem um gestor responsável. As chaves estrangeiras de **gestor**, **tecnico** e **backoffice** para **funcionario** garantem ainda que não existem registos de cargo sem um funcionário correspondente.

O identificador composto da ferramenta faz com que cada exemplar seja identificado pela chave composta (*codigo_tipo*, *nFerramenta*). A partir desta, a coluna *idFerramenta* é gerada automaticamente pela base de dados, disponibilizando assim um identificador único e legível que simplifica o seu uso na API sem introduzir redundância ao modelo.

A remoção lógica (*soft delete*) faz com que os funcionários nunca sejam completamente removidos da base de dados. São marcados como **inativos** (*ativo* = FALSE).

Desta maneira, preserva-se todo o histórico de requisições e tarefas associado, mantendo a rastreabilidade.

As vistas para leitura foram criadas para otimizar as consultas mais frequentes da API e evitar a repetição de junções complexas em múltiplos pontos do código. As vistas implementadas e os respectivos propósitos operacionais são apresentados de seguida:

- **View_Inventário_Detalhado:** Junta a ferramenta com o seu respetivo tipo, apresentando o inventário completo, incluindo nome, categoria, estado, disponibilidade e armário.
- **View_Email:** Associa o funcionário às tabelas de cargo, devolvendo a respetiva função do utilizador numa única coluna.
- **View_Mapa_Emprestimos:** Gera um mapa de empréstimos contendo o técnico, a ferramenta e as datas, servindo de base principal para listar o histórico e apurar as ferramentas em falta.
- **View_Ferramentas_Tecnico:** Indica as ferramentas que cada técnico tem atualmente em uso, isto é, requisições ainda por devolver, juntando os dados da requisição, da ferramenta e do tipo de ferramenta.
- **View_Tarefas_Detalhada:** Apresenta as tarefas detalhadas com o técnico atribuído e as ferramentas permitidas, incluindo título, estado e prioridade, funcionando como base para o *endpoint* de listagem de tarefas.
- **View_Tecnicos_Disponibilidade:** Fornece a lista de técnicos ativos com indicação imediata da sua disponibilidade, sendo marcado como "livre" caso não tenha tarefas por concluir.
- **View_Ferramentas_Reservadas_Tecnico:** Identifica as ferramentas reservadas para cada técnico através das tarefas ativas por concluir, tendo ferramentas permitidas por tarefa.

4.2 API REST

A comunicação entre a aplicação móvel e a componente de servidor é realizada através de uma API REST, desenvolvida, em Kotlin com a *framework* **Ktor** e com acesso aos dados via JDBC. Esta API funciona como camada intermédia entre a aplicação móvel e a base de dados, disponibilizando os dados necessários à interface e executando as regras de negócio associadas às operações realizadas pelos utilizadores.

A API organiza as suas rotas sob o prefixo `/api`, troca dados em formato JSON e devolve códigos de estado HTTP adequados ao resultado de cada operação.

As operações com vários passos, como criar uma requisição, atribuir uma tarefa, criar um funcionário, devolver uma ferramenta ou alterar o cargo de um funcionário, são executadas dentro de transações. Caso ocorra algum erro durante uma dessas operações, é feito *rollback*, evitando que a base de dados fique num estado parcialmente atualizado.

A lista completa dos *endpoints* desenvolvidos pode ser consultada no Apêndice C. A Tabela 4.1 destaca apenas as rotas principais que contêm lógicas de negócio mais complexas e que justificam uma análise detalhada.

Tabela 4.1: Principais pontos de acesso (*endpoints*) da API

Método	Rota	Descrição
POST	/api/requisicoes	Técnico requisita uma ferramenta.
POST	/api/tarefas	Gestor atribui uma tarefa e reserva as ferramentas permitidas.
PATCH	/api/requisicoes/{id}/devolver	Regista a devolução de uma requisição.
PATCH	/api/ferramentas/{id}/estado	Marca o estado de uma ferramenta.
PATCH	/api/funcionarios/{id}/cargo	Altera o cargo (permissões) de um funcionário.
PATCH	/api/funcionarios/{id}/desativar	Desativa um funcionário (<i>soft delete</i>).

Alguns comportamentos que se destacam nestes *endpoints*:

/api/requisicoes (POST) só cria a requisição se o técnico tiver uma tarefa não concluída que autorize aquela ferramenta; caso contrário, retorna **403 Forbidden**. Em caso de sucesso, cria a requisição, associa a ferramenta e marca **disponibilidade = "Requisitada"**.

/api/tarefas (POST) reserva as ferramentas de forma atômica: só reserva as que ainda estão "Disponível" (passando-as a "Reservada"); se alguma já não estiver disponível, faz *rollback* de toda a tarefa e retorna **409 Conflict**.

/api/requisicoes/{id}/devolver (PATCH) repõe a ferramenta como "Disponível", exceto se o seu estado for "Danificada", caso em que passa a "Em Manutencao".

/api/ferramentas/{id}/estado (PATCH) só permite mudar o estado se a ferramenta estiver requisitada; caso contrário, retorna **409 Conflict**.

/api/funcionarios/{id}/cargo (PATCH) remove o funcionário das tabelas de cargo antigas e insere-o na nova, validando que o cargo é GESTOR, TECNICO ou BACKOFFICE.

/api/funcionarios/{id}/desativar (PATCH) faz *soft delete* (**ativo = FALSE**), mantendo o funcionário no histórico em vez de o apagar.

4.3 Aplicação móvel multiplataforma

Na sequência das opções de desenvolvimento apresentadas no Capítulo 3, a aplicação móvel foi desenvolvida em Kotlin Multiplatform (KMP), com a interface construída em Compose Multiplatform. A lógica partilhada e a interface gráfica estão centralizadas no

módulo principal (`composeApp`), enquanto os conjuntos de código `androidMain` e `iosMain` ficam reservados às integrações que dependem do sistema operativo, como o *Firebase* e o *Google Sign-In*. A interface da aplicação foi desenvolvida como referência a identidade visual da TAP, de modo a aproximar o protótipo do contexto operacional considerado no projeto

4.3.1 Estrutura do Projeto

Para tirar partido da abordagem multiplataforma, o projeto foi organizado em diferentes conjuntos de código (*source sets*), permitindo separar o código comum da aplicação das partes específicas de Android e iOS. Esta organização facilita a reutilização de código, reduz a duplicação e permite manter as integrações dependentes da plataforma nos módulos adequados.

A Figura 4.2 apresenta a organização dos principais *source sets* utilizados no projeto, distinguindo o código comum da aplicação das implementações específicas de cada plataforma.

androidMain

- Contém as implementações específicas da plataforma Android, nomeadamente as integrações com *Firebase* e *Google Sign-In*.

iosMain

- Contém as implementações específicas da plataforma iOS, garantindo que as funcionalidades dependentes do sistema operativo são tratadas fora do código comum.

`androidMain`

- Contém as implementações específicas da plataforma Android, nomeadamente as integrações com *Firebase* e *Google Sign-In*.

commonMain

- **core**: Código transversal à aplicação, subdividido de forma modular por domínio de negócio (como `alerta`, `armario`, `auth`, `ferramenta` e `network`).
- **di**: Centraliza as dependências do projeto através do ficheiro `AppModule.kt`.
- **feature**: Agrupa o código por funcionalidade, isolando a lógica e os ecrãs de fluxos específicos, como `backoffice`, `gestor`, `tecnico`, etc.
- **ui**: Camada visual genérica e partilhada, que inclui as configurações visuais (*theme*) e os componentes globais reutilizáveis (`Components.kt`).
- **App.kt**: Ficheiro posicionado na raiz do pacote que atua como o ponto de entrada da interface e gestor central das rotas de navegação.

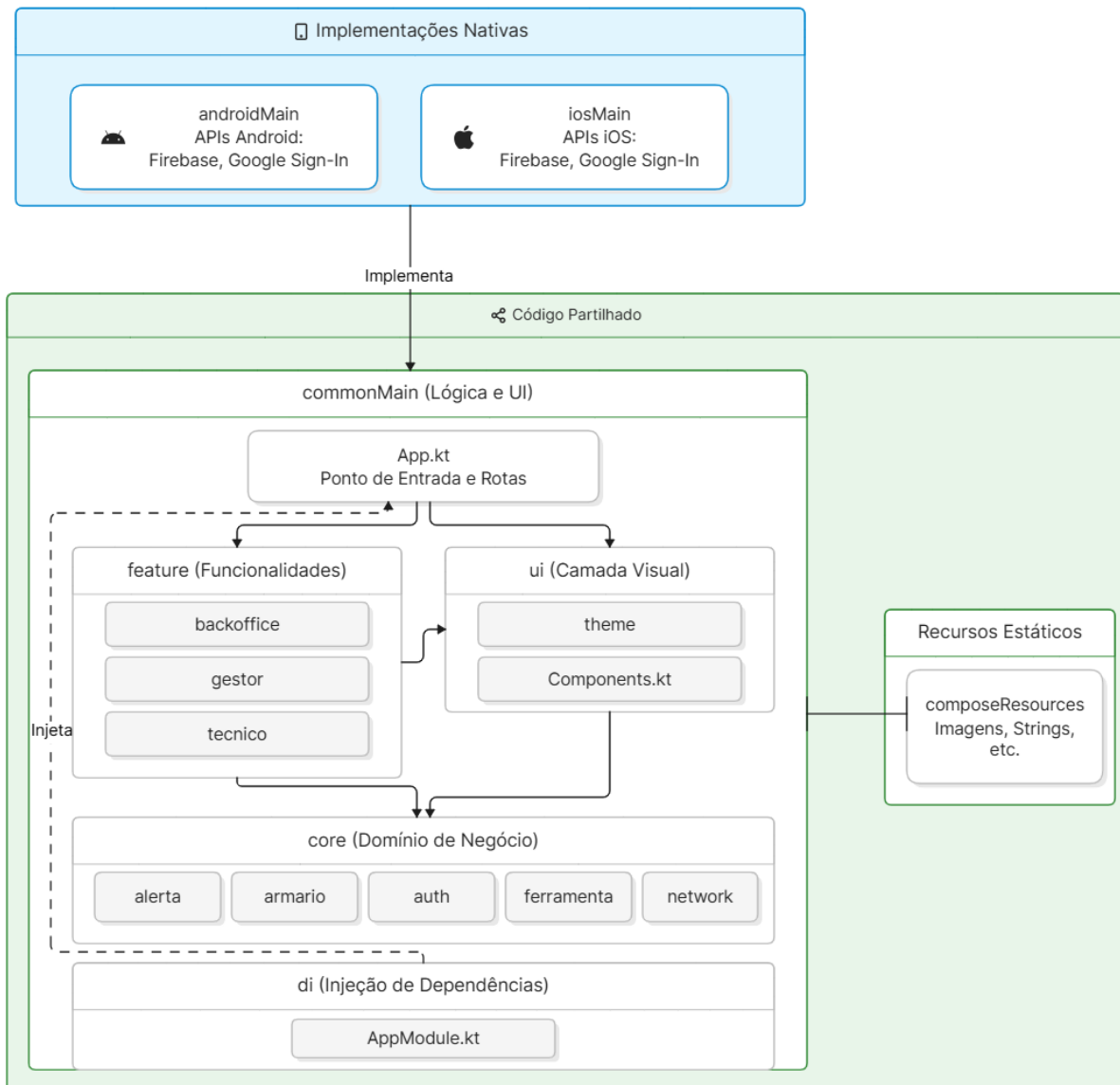


Figura 4.2: Diagrama de *Source Sets* utilizados

Recursos Estáticos

- Contém os recursos não executáveis usados pela aplicação, como imagens, ícones, fontes, ficheiros de configuração visual ou outros elementos necessários à construção da interface. Estes recursos são mantidos separados do código-fonte, facilitando a sua reutilização e gestão ao longo do projeto.

4.3.2 Aplicação do padrão MVVM

Na implementação da aplicação móvel, o padrão MVVM, apresentado no Capítulo 3, foi aplicado de forma sistemática aos diferentes ecrãs, sendo cada ecrã da aplicação suportado por um *ViewModel* próprio. O estado exposto à interface é tipado como *StateFlow* imutável, por exemplo `state: StateFlow<LoginUiState>`, sendo o estado interno mutável (`_state`) alterado apenas dentro do *ViewModel*, como `onEmailChange`,

`onLoginClick`, `onGoogleToken`, com as operações assíncronas a serem executadas em `viewModelScope.launch`.

O estado de cada ecrã é modelado como uma *data class* que agrupa toda a informação necessária à interface, sendo recriado com `copy()` a cada alteração.

4.3.3 Construção de um Ecrã Compose

Ao contrário da abordagem tradicional do Android, baseada em ficheiros *Extensible Markup Language* (XML), no **Compose** um ecrã é uma função anotada com `@Composable` que descreve o aspeto da interface para um determinado estado. A árvore de componentes nunca é alterada manualmente: quando o estado muda, a função volta a ser executada através do processo de recomposição e a *framework* atualiza apenas o que mudou. Na implementação dos ecrãs da aplicação, esta abordagem foi concretizada através de quatro aspetos principais:

- **Separação entre ecrã “com estado” e ecrã “sem estado”:** Cada ecrã é dividido em duas funções. A função externa (ex.: `LoginScreen(viewModel)`) liga-se ao *ViewModel*, obtém o estado com `collectAsState()` e encaminha-o. A função interna (ex.: `LoginScreenContent(...)`) é puramente visual: recebe apenas dados e funções de *callback* (`onEmailChange`, `onLoginClick`) e desconhece por completo o *ViewModel*, o *Firebase* ou o servidor.
- **Fluxo de dados unidirecional:** O estado desce do *ViewModel* para o `...Content` e os eventos sobem isto é, o ecrã comunica as ações do utilizador através de *lambdas* que invocam métodos do *ViewModel*.
- **Interface reativa:** A aparência é sempre função do estado, não havendo atualização manual de componentes. No botão de *login*, por exemplo, define-se `enabled = !isLoading` e o texto alterna com `if (isLoading) "A entrar..." else "Entrar"`; assim que o campo `isLoading` muda, o **Compose** reexecuta a função e o botão reflete automaticamente o novo estado.
- **Layout por composição e Modifier:** A interface é construída aninhando funções de *layout*, como `Column` para organização vertical, `Row` para organização horizontal e `Box` para sobreposição. Cada componente é ajustado através de uma cadeia de *Modifier*, como

```
.fillMaxSize(), .padding(24.dp), .background(...), .clickable{...},  
....
```
- **Listas eficientes com LazyColumn:** No ecrã de ferramentas, todo o conteúdo com deslocamento vertical é montado num único `LazyColumn`, combinando blocos únicos, como o título, a barra de pesquisa e os filtros, com listas repetidas, como `items(state.ferramentas) { ... }`. Apenas os elementos visíveis no ecrã são efetivamente compostos, o que é essencial para o desempenho quando o inventário é extenso.
- **Componentes reutilizáveis e tema:** Os ecrãs são compostos a partir de peças mais pequenas, específicas, como `FerramentaItemCard` e `TemplateCard` ou partilhadas em `Components.kt`, como `TopBar`, `BarraPesquisa` e `FilterChip`. As cores

e a tipografia provêm de um tema central (`AppTheme/MaterialTheme`), definido uma única vez em `Theme.kt`, garantindo coerência visual com a identidade gráfica definida para o protótipo em toda a aplicação.

4.3.4 Navegação

Cada destino de navegação é representado por um objeto de dados `@Serializable`, por exemplo `GestorRoute` ou `TecnicoRoute`, definido em `Routes.kt`. O controlador principal é o `NavHost`, configurado em `App.kt`.

O fluxo de ecrãs adapta-se dinamicamente ao perfil do utilizador. Através da função `routeForRole`, o sistema avalia o cargo do funcionário autenticado e direciona-o automaticamente para o seu painel inicial adequado. Desta forma, um Técnico, um Gestor ou um utilizador de *Back-Office* são encaminhados para áreas distintas da aplicação, de acordo com as funcionalidades associadas ao respetivo perfil.

Além disso, após um início de sessão com sucesso, o ecrã de *login* é removido do histórico, através de `popUpTo` com `inclusive=true`, impedindo que o utilizador regresse ao ecrã de autenticação ao usar a navegação de retorno do dispositivo.

4.3.5 Arranque e Gestão de Sessão

O arranque da aplicação foi desenhado para ser fluido e automático. Quando a aplicação é iniciada, o sistema verifica de imediato se o utilizador já tem sessão iniciada, utilizando o `SessionViewModel`.

Enquanto esta validação ocorre em segundo plano, a aplicação apresenta um ecrã de carregamento (`SplashScreen`). Assim que a verificação termina, o encaminhamento é feito automaticamente, de acordo com o estado da sessão.

- **Com sessão (*Authenticated*):** Se já existir uma sessão guardada no dispositivo móvel, o ecrã de *login* é completamente ignorado e o utilizador é levado diretamente para o painel principal do seu cargo (persistência).
- **Sem sessão (*NoSession*):** Caso contrário, o utilizador é direcionado para a página inicial da aplicação para iniciar o processo de autenticação.

Esta organização permite reduzir passos desnecessários no arranque da aplicação e garantir que o utilizador é encaminhado para o ecrã adequado em função do seu estado de autenticação.

4.3.6 Autenticação

O sistema suporta acesso por email e palavra-passe e autenticação federada através de Google SSO. Este sistema de *Single Sign-On* foi incluído como uma funcionalidade experimental para que, possa ser avaliada a possibilidade de funcionários de organizações parceiras acederem à aplicação usando mecanismos de autenticação federada. A lógica central está definida no ficheiro `Contrato.kt` através da interface `AuthRepository`. Esta interface estabelece as operações base (`loginEmail`, `loginGoogle`, `getSession`, `logout`) e gere os possíveis erros de forma segura utilizando classes seladas (`AuthResult` e `AuthError`).

É importante destacar que o *Firebase* atua apenas na validação de identidade. O cargo real do utilizador (Gestor, Técnico ou *Back-Office*) provém exclusivamente

da componente de servidor. Após um início de sessão com sucesso, a classe `FuncionarioRemoteDataSource` contacta a API (`/api/funcionarios/email`) para obter o perfil e o turno do funcionário.

A base de código partilhada apenas reconhece a interface genérica, obrigando cada plataforma a fornecer a sua implementação real: o `androidMain` implementa o `FirebaseAuthRepository` com ajuda do `GoogleSignInHelper`, enquanto o `iosMain` injeta o `iOSAuthRepository`.

4.3.7 Injeção de Dependências e Acesso à API

A organização das dependências do projeto está centralizada no ficheiro `AppModule`. Para evitar a criação imediata de objetos mais pesados no arranque da aplicação, usou-se a técnica `by lazy`, fazendo com que estes recursos sejam instanciados apenas na primeira vez que são necessários e sejam, sendo depois reutilizados.

A comunicação com o servidor é feita através do `Ktor`, que está configurado para realizar a serialização e desserialização automática dos dados trocados com a API em formato `JSON`.

Um detalhe importante é a gestão do endereço do servidor (`baseUrl`). Como um emulador Android, um simulador iOS e um dispositivo físico real acedem à rede local de formas diferentes, usou-se o mecanismo `expect/actual`, permitindo que a base de código partilhada delegue em cada plataforma a responsabilidade de fornecer o endereço correto do servidor

Capítulo 5

Validação e Testes

Este capítulo apresenta a validação prática do sistema *SmartTool Cabinet* através da simulação dos seus fluxos operacionais. O conteúdo está estruturado em quatro partes: inicialmente, validam-se os mecanismos de autenticação e acesso; de seguida, analisa-se o fluxo de atribuição e requisição de ferramentas; posteriormente, demonstra-se o processo de devolução e reporte de anomalias; e, por fim, reflete-se sobre a relevância de futuros testes de usabilidade junto dos profissionais da TAP.

5.1 Metodologia de validação

A validação do protótipo foi realizada através da simulação dos principais fluxos de utilização da aplicação móvel. Estes fluxos foram definidos com base nos requisitos apresentados no Capítulo 2 e representam as operações mais importantes do sistema no contexto da gestão de ferramentas em manutenção aeronáutica.

Para organizar os testes, foram considerados os três perfis de utilizador da aplicação: Técnico, Gestor e *Back-Office*. Em cada caso, foi definido um cenário de utilização, identificando o objetivo do teste, as ações realizadas na aplicação e o resultado obtido. Desta forma, foi possível verificar se a aplicação respondia corretamente às operações previstas para cada perfil.

Os testes incidiram sobretudo sobre a autenticação e o encaminhamento do utilizador para o painel correspondente ao seu cargo, a atribuição de tarefas pelo Gestor, a requisição de ferramentas pelo Técnico, a devolução de ferramentas e o reporte de anomalias. Estes fluxos foram escolhidos por corresponderem às funcionalidades centrais do protótipo.

Os testes apresentados nas secções seguintes foram realizados com dados de demonstração, criados para representar funcionários, cargos, ferramentas, armários, tarefas e requisições. Assim, a validação realizada incide sobre o comportamento funcional do protótipo em ambiente de teste, não correspondendo ainda a uma validação em contexto operacional real.

5.2 Autenticação e Controlo de acessos

O primeiro passo para a validação do sistema consiste em verificar o mecanismo de controlo de acessos.

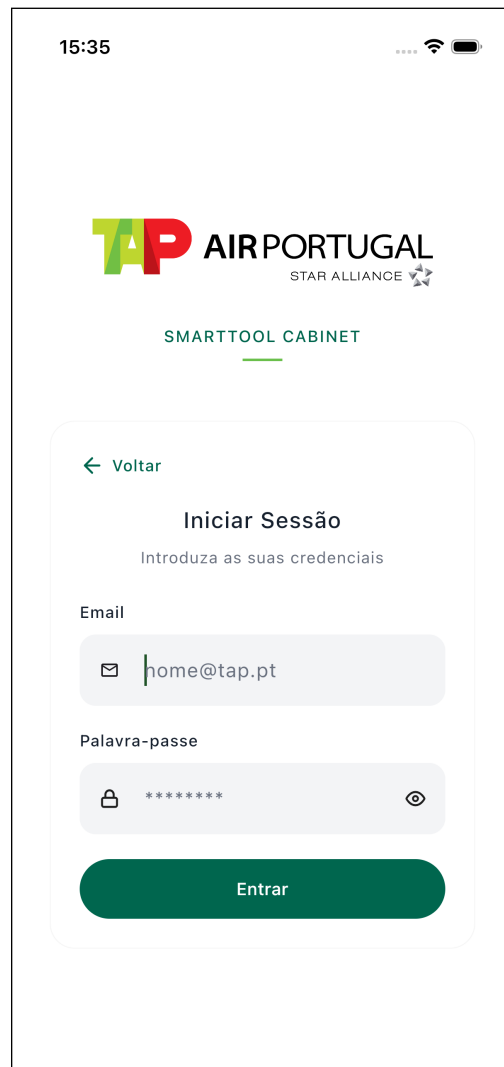
Como ilustrado na Figura 5.1a, o utilizador é recebido pelo ecrã inicial de entrada, onde é apresentada a opção de utilizar a autenticação federada ou prosseguir com credenciais

tradicionais. Caso opte pela via tradicional, a Figura 5.1b demonstra a interface de inserção de *email* e palavra-passe.

Os testes realizados confirmam que a aplicação bloqueia o acesso sem autenticação e comunica de forma segura com os serviços de autenticação.



(a) Ecrã inicial de autenticação



(b) Ecrã de autenticação com *email* e palavra-passe

Figura 5.1: Fluxo de autenticação da aplicação

5.3 Atribuição de tarefas e requisição de ferramentas

O teste principal ao fluxo de trabalho operacional começa no perfil do Gestor. Este teste teve como objetivo verificar se o Gestor consegue atribuir tarefas e ferramentas a um Técnico e se, posteriormente, o Técnico consegue consultar e requisitar as ferramentas associadas. A Figura 5.2 apresenta o ecrã de gestão de tarefas, onde é possível observar a atribuição de novas intervenções, como a manutenção do Motor A330, a técnicos específicos, com as respetivas ferramentas. No topo do ecrã é possível visualizar os alertas

ativos, que correspondem a ferramentas em falta, armários destrancados, etc.

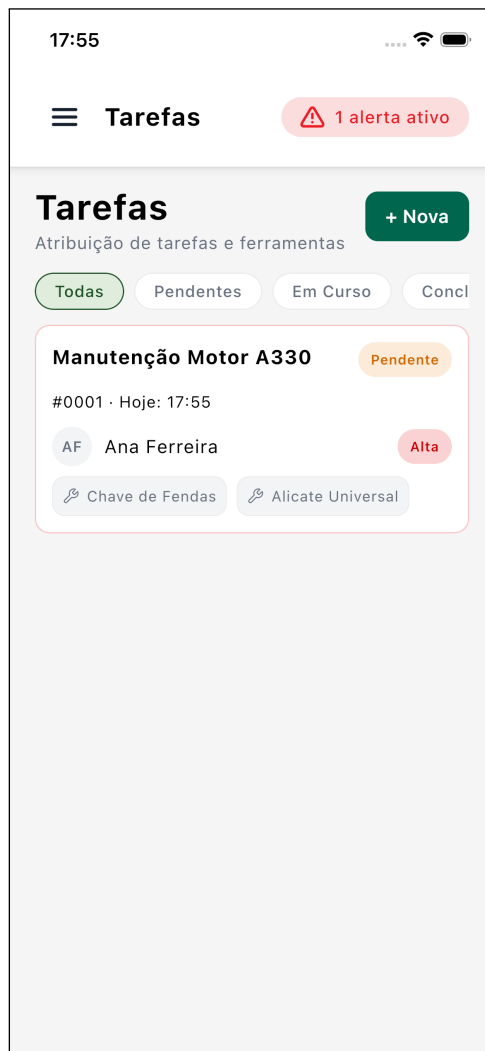
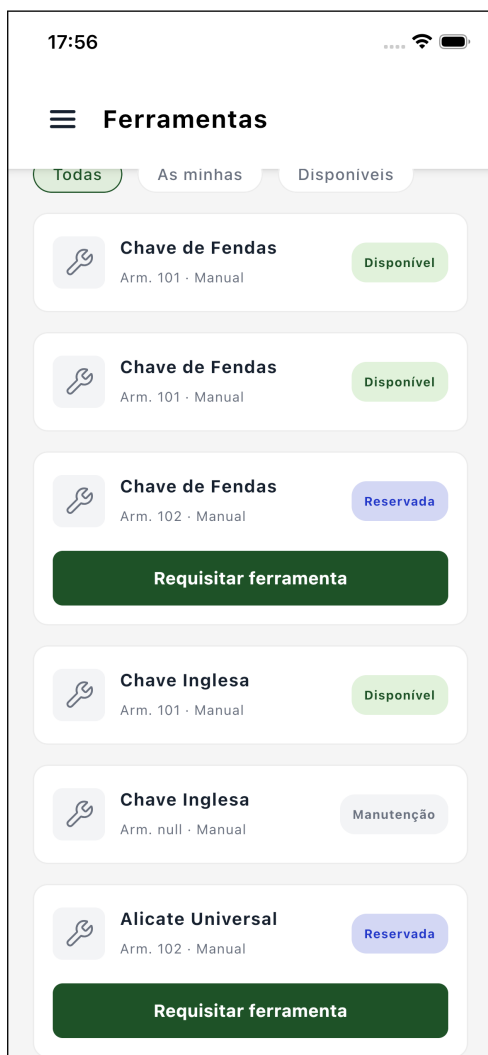


Figura 5.2: Atribuição de tarefa e ferramentas pelo Gestor

Do lado do Técnico, o processo continua no ecrã de ferramentas. A Figura 5.3a mostra a interface onde o utilizador, com perfil de Técnico, vê as ferramentas com estado "Disponível" e as pode levantar através do botão de requisição. Depois de confirmar, o estado da ferramenta é atualizado na base de dados e passa a "Em Uso", como se vê na Figura 5.3b. Os testes realizados mostram que a aplicação atualiza o estado da ferramenta em tempo real, permitindo saber sempre quem a tem.



(a) Ação de requisitar uma ferramenta disponível



(b) Ferramentas com estado atualizado para Em Uso

Figura 5.3: Processo de requisição de ferramentas por um Técnico

5.4 Devolução de ferramentas e Reporte de anomalias

A última fase do ciclo é o fim da tarefa e a devolução do material. Nesta etapa, valida-se se o Técnico consegue devolver uma ferramenta em condições normais ou assinalar que esta se encontra em mau estado.

No painel das ferramentas em posse do funcionário (Figura 5.4), há duas opções: devolver normalmente ou reportar mau estado. Ao simular o reporte de mau estado, o sistema muda o estado da ferramenta para “Manutenção”, impedindo que esta volte a ser considerada disponível para novas requisições.

Desta forma, o teste confirma que a aplicação suporta o encerramento do ciclo de utilização da ferramenta e que o estado registado no sistema é atualizado de acordo com a situação indicada pelo Técnico.

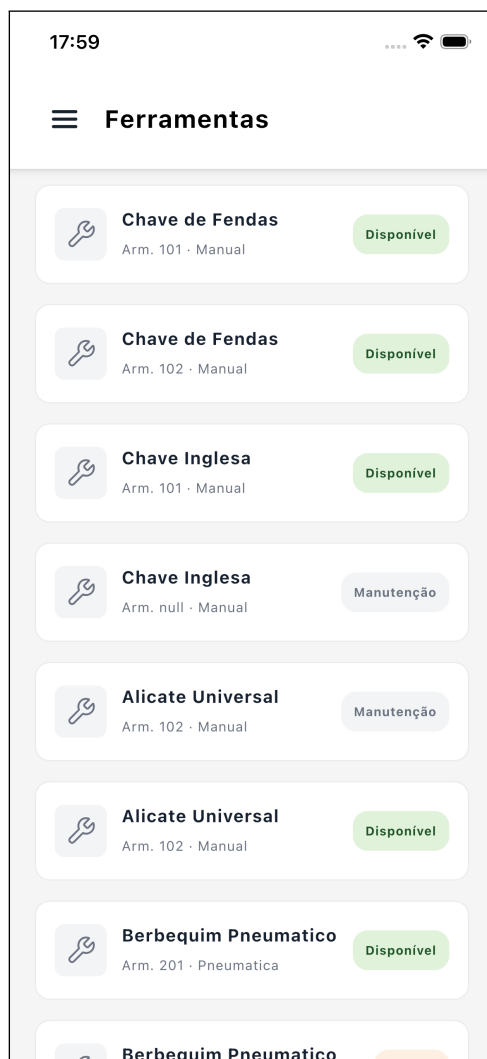


Figura 5.4: Processo de devolução e reporte de anomalia

Capítulo 6

Conclusões e Trabalho Futuro

A gestão de ferramentas em contexto de manutenção aeronáutica exige mecanismos rigorosos de controlo e rastreabilidade. A possibilidade de uma ferramenta ficar esquecida numa aeronave, ou de não ser possível reconstruir o seu histórico de utilização, representa um risco operacional relevante. Neste contexto, o trabalho desenvolvido teve como objetivo criar uma solução digital que apoiasse a gestão centralizada de ferramentas, reduzindo a dependência de processos manuais e facilitando o acompanhamento das operações de requisição, devolução e controlo do estado das ferramentas.

Como resultado, neste projeto foi desenvolvido o sistema informático *SmartTool Cabinet*, centrado numa aplicação móvel multiplataforma para Android e iOS. A aplicação contempla três perfis de utilizador — Técnico, Gestor e *Back-Office* — e permite organizar as funcionalidades de acordo com as responsabilidades de cada um. O Técnico pode consultar tarefas, requisitar ferramentas, devolver material e reportar anomalias. O Gestor pode atribuir tarefas, associar ferramentas, consultar alertas, verificar ferramentas em falta e acompanhar o estado das ferramentas e dos armários. O *Back-Office* pode gerir funcionários, níveis de acesso e turnos.

Para suportar o funcionamento da aplicação móvel, foi também desenvolvida uma componente de servidor em *Ktor*, ligada a uma base de dados *MySQL* e disponibilizada através de uma *API REST*. Esta componente funciona como infraestrutura de suporte ao protótipo, centralizando os dados e aplicando as regras de negócio necessárias ao funcionamento do sistema. A aplicação móvel foi desenvolvida em *Kotlin Multiplatform*, com interface em *Compose Multiplatform*, seguindo uma organização baseada no padrão *MVVM*, navegação *type-safe*, injeção de dependências e autenticação por email e palavra-passe ou autenticação federada.

A validação realizada permitiu testar os principais fluxos de interação da aplicação, nomeadamente a autenticação e o controlo de acessos, a atribuição de tarefas e a requisição de ferramentas, e ainda a devolução de ferramentas e o reporte de anomalias. Estes testes permitiram verificar o funcionamento dos principais fluxos do protótipo em ambiente de teste, com dados de demonstração. No entanto, a solução não foi ainda testada em contexto operacional real, nem avaliada com utilizadores finais.

Apesar dos resultados obtidos, a validação realizada decorreu apenas em ambiente de teste, com dados de demonstração. Não foram realizados testes de escala, que permitissem avaliar o comportamento do sistema com um número elevado de utilizadores, ferramentas, armários, tarefas e requisições. Também não foram realizados testes de usabilidade com utilizadores finais, pelo que não foi possível avaliar, num contexto próximo do real, a clareza da interface, a adequação dos fluxos de navegação e a facilidade de execução

das principais operações. Além disso, a componente de servidor desenvolvida teve como objetivo suportar o protótipo funcional, podendo a evolução futura do sistema exigir alterações à API, nomeadamente para suportar a integração com outros sistemas ou com armários físicos.

Como trabalho futuro, identificam-se as seguintes ideias para uma evolução do sistema:

- Realização de testes de usabilidade em contexto real, com Técnicos, Gestores e elementos de *Back-Office*, para avaliar a clareza da interface, a facilidade de utilização e a adequação dos fluxos de navegação;
- Integração com os armários físicos, ligando a aplicação ao *hardware* real, como fechaduras eletrónicas e mecanismos de identificação automática, nomeadamente RFID);
- Implementação completa do sistema de notificações, com o envio de alertas de grau de importância elevado ao Gestor;
- Integração com os sistemas de informação já existentes em organizações;
- Concretização dos requisitos de acessibilidade previstos, como o modo para utilizadores daltónicos.
- Realização de testes de escala, de modo a avaliar o comportamento do sistema com um maior número de utilizadores, ferramentas, armários, tarefas e requisições;

Em síntese, o trabalho desenvolvido permitiu demonstrar a viabilidade de uma aplicação móvel multiplataforma para apoiar a gestão centralizada de ferramentas em manutenção aeronáutica. O protótipo implementado suporta os principais perfis de utilizador e os fluxos essenciais de atribuição, requisição e devolução de ferramentas, constituindo uma base funcional para evoluções futuras do sistema.

Referências bibliográficas

- [Android Developers, 2024] Android Developers (2024). Guia para a arquitetura de apps. <https://developer.android.com/topic/architecture?hl=pt-br>. Acedido em 4 de julho de 2026.
- [CTT — Correios de Portugal, 2026] CTT — Correios de Portugal (2026). Cacifos CTT. <https://www.ctt.pt/particulares/receber/locais-de-entrega-alternativos/cacifos/index>. Acedido em 7 de julho de 2026.
- [del Carmen León-Araujo et al., 2019] del Carmen León-Araujo, M., Gómez-Inhieto, E., e Acaiturri-Ayesta, M. T. (2019). Implementation and evaluation of a rfid smart cabinet to improve traceability and the efficient consumption of high cost medical supplies in a large hospital. *Journal of Medical Systems*, 43(178).
- [Fielding, 2000] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine. Ph.D. dissertation.
- [Gabriels et al., 2016] Gabriels, R., Oni, O., Oshin, O., Idachaba, F., e Nkordeh, N. (2016). Development and implementation of a smart medical cabinet using rfid technology in developing countries. In *Proceedings of the World Congress on Engineering 2016 Vol I (WCE 2016)*, London, U.K.
- [GeeksforGeeks, 2025] GeeksforGeeks (2025). Difference between MVC, MVP and MVVM architecture pattern in Android. Acedido em: 7 de julho de 2026.
- [Google, 2026] Google (2026). Firebase. <https://firebase.google.com/>. Acedido em 7 de julho de 2026.
- [Hamzah et al., 2021] Hamzah, R., Ahmad Tarmizi, N. I. A., Zahari, N. A., e Shamsudin, I. I. (2021). The development of smart ceiling cabinet. *Journal of Civil Engineering Technology*, 1(1):17–27.
- [JetBrains, 2026a] JetBrains (2026a). Compose Multiplatform. <https://kotlinlang.org/compose-multiplatform/>. Acedido em 7 de julho de 2026.
- [JetBrains, 2026b] JetBrains (2026b). Kotlin Multiplatform. <https://kotlinlang.org/multiplatform/>. Acedido em 7 de julho de 2026.
- [JetBrains, 2026c] JetBrains (2026c). Ktor. <https://ktor.io/>. Acedido em 7 de julho de 2026.

- [Kotlin Documentation, 2024] Kotlin Documentation (2024). Sealed classes and interfaces. <https://kotlinlang.org/docs/sealed-classes.html>. Acedido em 4 de julho de 2026.
- [Oracle Corporation, 2026] Oracle Corporation (2026). MySQL. <https://www.mysql.com/>. Acedido em 7 de julho de 2026.
- [Pal Amutha et al., 2012] Pal Amutha, K., Sethukkarasi, C., e Pitchiah, R. (2012). Smart kitchen cabinet for aware home. In *SMART 2012*, Chennai, India. National Ubiquitous Computing Research Centre, Centre for Development of Advanced Computing.
- [Trigo et al., 2026] Trigo, P., Filipe, P., e Teófilo, A. (2026). JDBC. pdf. Slides da Unidade Curricular Sistema de Base de Dados 25/26.

Apêndice A

Gestão de Versões e Repositório

Este apêndice documenta a forma como o grupo organizou o trabalho colaborativo e a gestão das várias versões do projeto.

A.1 Ferramentas e Repositório

O código do projeto foi gerido com Git e está centralizado num repositório no *GitHub*, acessível em:

<https://github.com/lulssam/SmartTool-Cabinet>

Ao todo, o desenvolvimento traduziu-se em mais de 200 *commits*, refletindo o carácter incremental da abordagem seguida. A utilização de uma plataforma remota permitiu que os dois elementos do grupo trabalhassem em simultâneo, mantendo sempre uma cópia sincronizada e um histórico completo de todas as alterações.

A.2 Fluxo de Trabalho

O grupo adotou um modelo de desenvolvimento baseado em ramos (*branches*). O ramo principal (**main**) foi mantido sempre num estado estável e funcional, servindo como versão de referência do projeto.

Cada nova funcionalidade ou correção foi desenvolvida num ramo próprio, isolando o trabalho em curso do ramo principal. Depois de concluída e testada, essa alteração era integrada no **main** através de um *pull request* (PR), permitindo controlar a integração das alterações antes da sua junção (*merge*).

A.3 Convenção de Nomes dos Ramos

Os ramos foram nomeados de forma descritiva, identificando a funcionalidade em desenvolvimento e, frequentemente, o perfil de utilizador a que dizia respeito. Esta convenção facilitou a organização do trabalho e a leitura do histórico. Alguns exemplos:

- **gesto-adicionar-tarefa**: funcionalidades do perfil de Gestor;
- **tecnico-requisitar-debug**, **Tecnico-Historico**: funcionalidades do perfil de Técnico;

- `bo-hamburger`, `bo-debugging`: funcionalidades do perfil de *Back-Office*;
- `Tarefas-resolver`: correções transversais.

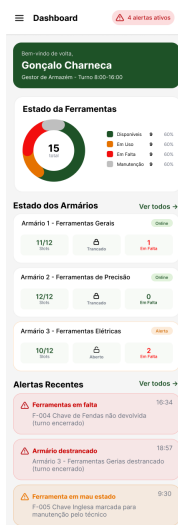
Esta separação por ramos, aliada à integração via *pull requests*, garantiu um desenvolvimento organizado e colaborativo, minimizando conflitos e preservando a estabilidade do ramo principal ao longo de todo o projeto.

Apêndice B

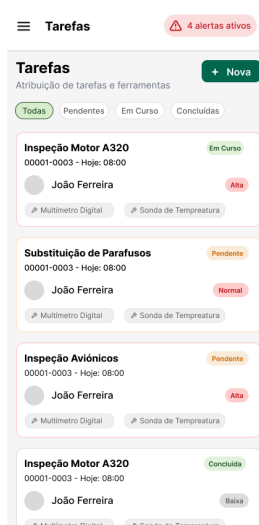
Protótipo da Interface (Mockups)

Este apêndice apresenta os principais *mockups* da aplicação, desenvolvidos na fase de prototipagem com a ferramenta *Figma*. Os ecrãs estão organizados pelos três perfis de utilizador do sistema: Gestor, Técnico e *Back-Office*.

B.1 Perfil de Gestor



(a) Painel principal



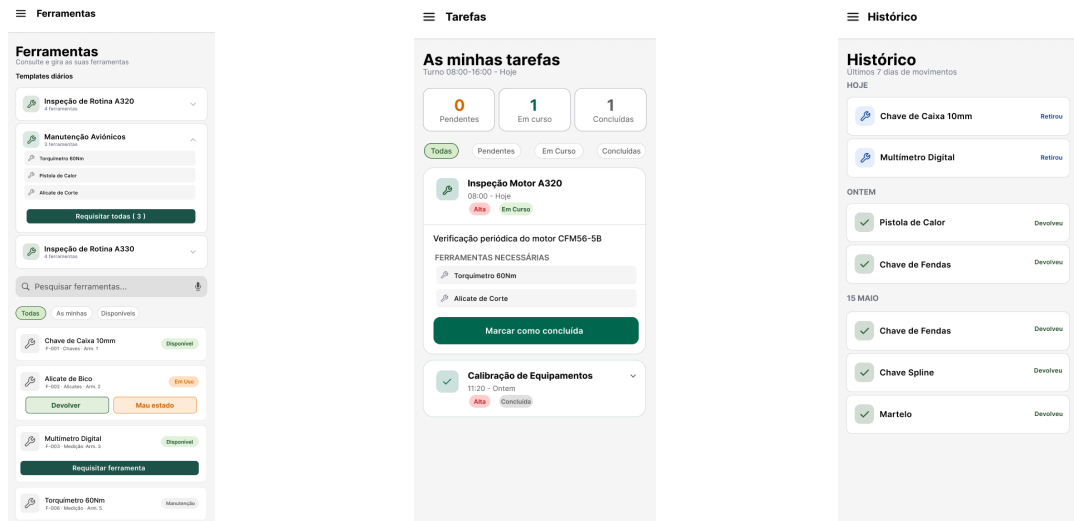
(b) Gestão de tarefas



(c) Estado dos armários

Figura B.1: *Mockups* do perfil de Gestor.

B.2 Perfil de Técnico



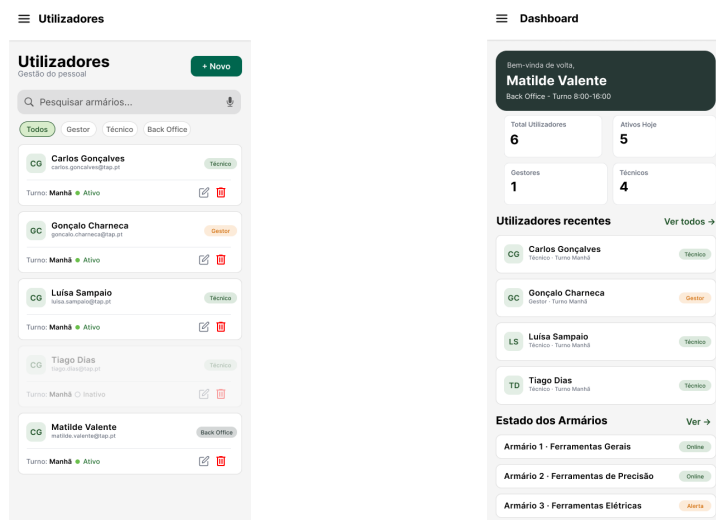
(a) Lista de ferramentas

(b) Lista de tarefas

(c) Histórico de requisições

Figura B.2: *Mockups* do perfil de Técnico.

B.3 Perfil de Back-Office



(a) Gestão de funcionários

(b) Painel principal

Figura B.3: *Mockups* do perfil de *Back-Office*.

Apêndice C

Tabela dos Pontos de Acesso

Este apêndice apresenta a lista completa dos principais pontos de acesso (endpoints) disponibilizados pela API REST desenvolvida para o protótipo. A tabela complementa a Secção 4.2, onde são analisadas apenas as rotas com lógica de negócio mais relevante.

Tabela C.1: Pontos de acesso (*endpoints*) da API

Método	Rota	Descrição
GET	/api/ferramentas	Lista o inventário detalhado de ferramentas.
GET	/api/ferramentas/em-falta	Ferramentas não devolvidas e respetivo detentor.
GET	/api/armarios	Lista de armários e o seu estado.
GET	/api/armarios/{id}/ferramentas	Ferramentas contidas num armário.
GET	/api/tarefas	Lista de tarefas com o técnico atribuído e as ferramentas permitidas.
GET	/api/tecnicos	Lista de técnicos e a sua disponibilidade.
GET	/api/tecnicos/{id}/ferramentas	Ferramentas que o técnico tem em uso (por devolver).
GET	/api/tecnicos/{id}/reservadas	Ferramentas reservadas ao técnico pelas suas tarefas.
GET	/api/funcionarios	Lista todos os funcionários (nome, cargo, turno, estado).
GET	/api/funcionarios/{email}	Dados do funcionário a partir do <i>email</i> .
GET	/api/historico	Requisições dos últimos 30 dias.
GET	/api/historico/tecnico/{id}	Histórico de requisições de um técnico (últimos 30 dias).
GET	/api/alertas	Alertas ativos (ferramentas em falta e armários destrancados).
POST	/api/requisicoes	Técnico requisita uma ferramenta.
POST	/api/tarefas	Gestor atribui uma tarefa e reserva as ferramentas permitidas.
POST	/api/funcionarios	Adiciona um funcionário (e o seu cargo).
PATCH	/api/requisicoes/{id}/devolver	Regista a devolução de uma requisição.
PATCH	/api/ferramentas/{id}/estado	Marca o estado de uma ferramenta.
PATCH	/api/funcionarios/{id}/cargo	Altera o cargo (permissões) de um funcionário.
PATCH	/api/funcionarios/{id}/turno	Define o turno de um funcionário.
PATCH	/api/funcionarios/{id}/desativar	Desativa um funcionário (<i>soft delete</i>).

Apêndice D

Modelo Entidade Associação

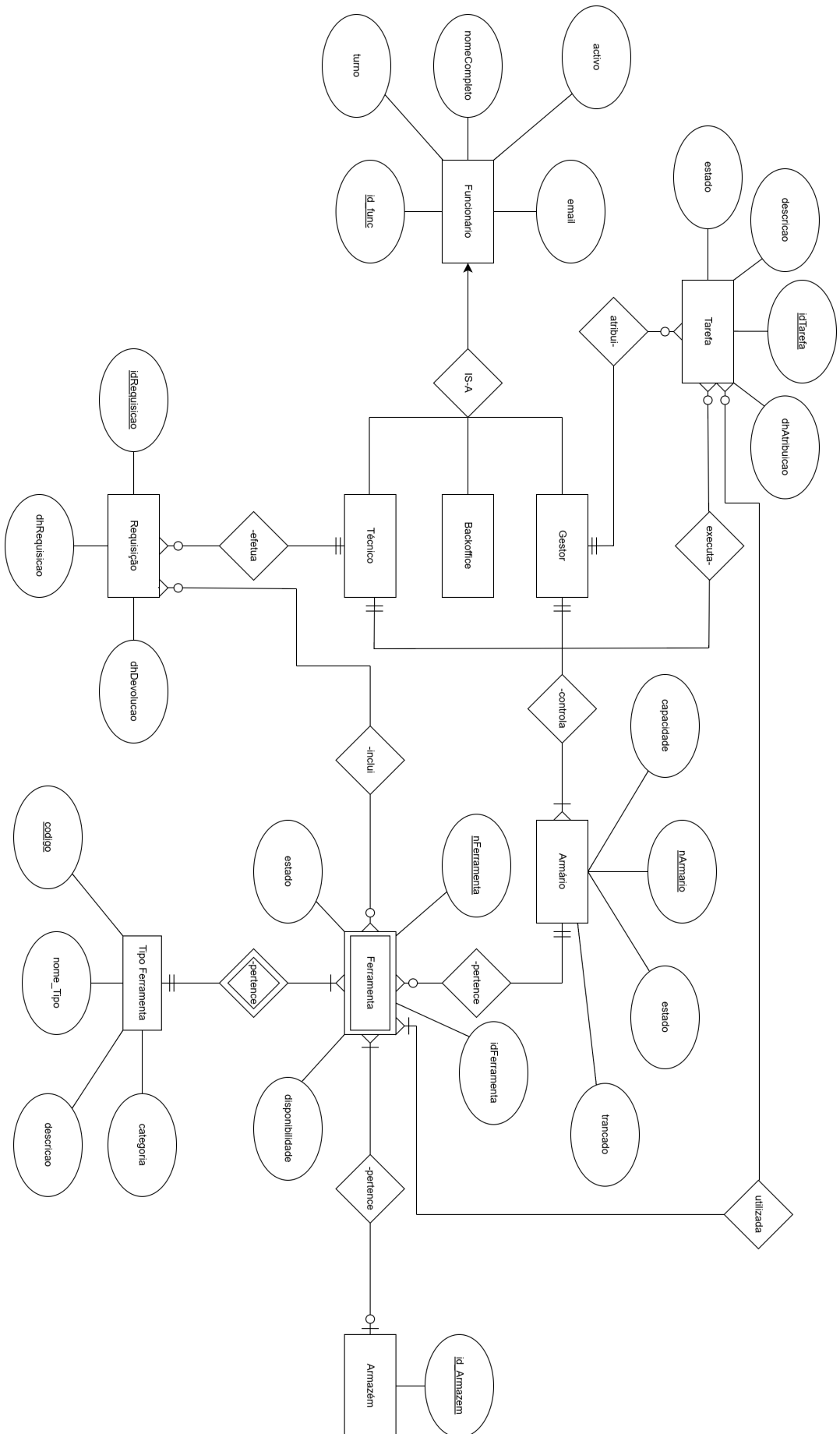


Figura D.1: Modelo Entidade-Associação da base de dados do sistema.